

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
TRUNG TÂM PHÁT TRIỂN CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN

Đề tài:

**XÂY DỰNG ỨNG DỤNG WEB HỖ TRỢ HỌC TẬP
CHUYỂN ĐỔI TÀI LIỆU THÀNH BỘ CÂU HỎI, THẺ
GHI NHỚ VÀ SƠ ĐỒ TƯ DUY.**

Giảng viên hướng dẫn: ThS. Mai Xuân Hùng

Lớp: IE207.F11.CN1.CNTT - Đồ án

Sinh viên thực hiện: Lê Hải Phúc

23730116

TP HCM, tháng 12 năm 2025

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

....., ngày.....tháng.....năm 2025

Người nhận xét

(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Mỗi thành công đạt được không chỉ đến từ nỗ lực cá nhân mà còn từ sự đồng hành và hỗ trợ, dù ít hay nhiều, dù trực tiếp hay gián tiếp. Trong suốt học kỳ vừa qua, quá trình thực hiện đồ án “Xây dựng ứng dụng web hỗ trợ học tập chuyển đổi tài liệu thành bộ câu hỏi, thẻ ghi nhớ và sơ đồ tư duy” là một hành trình vừa nhiều thử thách vừa nhiều trải nghiệm, với không ít lúc bối rối, loay hoay giữa ý tưởng, kiến thức và cách triển khai thực tế.

Trong hành trình đó, xin được gửi lời cảm ơn chân thành đến thầy Mai Xuân Hùng vì đã tận tâm hướng dẫn, gợi ý một cách rõ ràng, cũng như góp ý kỹ lưỡng từ cấu trúc nội dung, bố cục cho đến cách trình bày sao cho mạch lạc, dễ hiểu. Nhờ những góp ý và định hướng của thầy, đồ án từng bước được điều chỉnh, phát triển và hoàn thiện hơn so với ban đầu.

Sau cùng, kính chúc thầy Mai Xuân Hùng luôn dồi dào sức khỏe và thuận lợi trong công tác giảng dạy, tiếp tục đồng hành và thực hiện sứ mệnh cao đẹp của mình là truyền đạt kiến thức cho các thế hệ mai sau.

MỤC LỤC

MỞ ĐẦU.....	1
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	3
1.1. Giới thiệu chung.....	3
1.2. Lý do chọn đề tài.....	4
1.3. Mục tiêu của đề tài.....	5
1.3.1. Mục tiêu tổng quát.....	5
1.3.2. Mục tiêu cụ thể.....	5
1.4. Đối tượng phục vụ, phạm vi và giới hạn của đề tài.....	6
1.4.1. Đối tượng phục vụ.....	6
1.4.2. Phạm vi chức năng.....	7
1.5. Phương pháp thực hiện.....	7
1.6. Tổng quan công nghệ, kiến trúc và tổ chức mã nguồn.....	10
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	10
2.1. Nền tảng kiến trúc ứng dụng web hỗ trợ học tập.....	10
2.2. Cơ sở lý thuyết về mô hình ngôn ngữ lớn và OpenAI Chat Completions.....	11
2.2.1. Tổng quan mô hình ngôn ngữ lớn (LLM).....	11
2.2.2. Thiết kế prompt và JSON schema cho nội dung học tập.....	12
2.2.3. Quản lý ngữ cảnh và giới hạn độ dài văn bản.....	13
2.3. Nền tảng dữ liệu văn bản và trích xuất từ tệp PDF/TXT.....	14
2.4. Cơ sở lý thuyết về các dạng nội dung học tập sinh ra từ tài liệu.....	15
2.4.1. Tóm tắt tài liệu.....	16
2.4.2. Sơ đồ tư duy (mindmap) và Mermaid.....	16
2.4.3. Flashcard hỏi – đáp.....	17
2.4.4. Quiz trắc nghiệm, đúng/sai và điền khuyết.....	17
2.5. Các thư viện chính và cơ chế xử lý lỗi, bảo mật.....	18
2.6. Tóm tắt chương.....	20
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	21
3.1. Phân tích yêu cầu.....	21
3.1.1. Yêu cầu chức năng.....	21
3.1.2. Yêu cầu phi chức năng.....	27
3.2. Mô hình tác nhân, use-case và luồng nghiệp vụ.....	29
3.2.1. Tác nhân hệ thống.....	29
3.2.2. Các use-case chính.....	30
3.2.3. Luồng hoạt động tổng quát.....	33
3.3. Thiết kế kiến trúc.....	35
3.3.1. Kiến trúc tổng thể 2 tầng.....	35
3.3.2. Luồng xử lý chi tiết phía backend.....	36

3.3.3. Sơ đồ triển khai (Deployment).....	38
3.4. Thiết kế dữ liệu & JSON schema.....	39
3.5. Thiết kế lớp, module và API.....	41
3.5.1. Các thành phần backend trong app.py.....	41
3.5.2. Các thành phần frontend trong script.js.....	42
3.6. Cấu trúc thư mục & tổ chức mã nguồn.....	43
3.7. Chuẩn mã hóa, xử lý lỗi & log.....	44
3.8. Ma trận truy vết yêu cầu.....	45
CHƯƠNG 4. THỰC NGHIỆM.....	46
4.1. Môi trường cài đặt và triển khai.....	46
4.1.1. Yêu cầu hệ thống.....	46
4.1.2. Chuẩn bị môi trường trên Windows (Cách 1 – dùng script .bat).....	47
4.1.3. Chuẩn bị môi trường thủ công (Windows / macOS / Linux).....	49
4.1.4. Kiểm tra khi khởi động & xử lý sự cố thường gặp.....	50
4.2. Thiết lập dữ liệu mẫu và kịch bản kiểm thử.....	52
4.2.1. Đặc điểm dữ liệu mẫu dùng trong thử nghiệm.....	52
4.2.2. Kịch bản kiểm thử.....	53
4.3. Kết quả thực nghiệm.....	58
4.3.1. Kết quả chức năng.....	58
4.3.2. Quan sát về hiệu năng.....	60
4.3.3. Kiểm chứng bảo mật & nhất quán dữ liệu.....	62
4.4. So sánh các kỹ thuật và đánh giá độ chính xác.....	64
4.4.1. Bảng so sánh: Gọi API AI vs kỹ thuật Text Ranking (không dùng AI).....	64
4.4.2. Đánh giá độ chính xác.....	66
4.5. Hướng dẫn sử dụng nhanh, những hình ảnh web thực tế.....	70
CHƯƠNG 5. KẾT LUẬN.....	78
5.1. Tổng kết đóng góp và giá trị của đề tài.....	78
5.1.1. Giá trị học thuật và phương pháp.....	78
5.1.2. Giá trị thực tiễn của ứng dụng.....	81
5.2. Hạn chế và hướng phát triển.....	83
5.2.1. Những hạn chế hiện tại.....	83
5.2.2. Hướng phát triển trong tương lai.....	84
5.3. Kết luận chung.....	86
TÀI LIỆU THAM KHẢO.....	88

DANH MỤC BẢNG VÀ HÌNH ẢNH

Bảng 3.1: Bảng liệt kê các yêu cầu chức năng.....	27
Bảng 4.1: Kịch bản kiểm thử chấp thuận.....	58
Bảng 4.2: Bảng đo thời gian.....	62
Bảng 4.3: Bảng so sánh giữa hai kỹ thuật.....	65
Bảng 4.4: đánh giá Summary.....	67
Bảng 4.5: đánh giá Flashcards.....	68
Bảng 4.6: đánh giá Quizzes.....	69
Hình 1.1: Sơ đồ quy trình xử lý nội dung học tập bằng AI.....	4
Hình 2.1: Mô tả schema.....	13
Hình 2.2: Giản lược ký tự.....	14
Hình 2.3: Cấu trúc schema của flashcard.....	17
Hình 2.4: Cấu trúc câu trắc nghiệm.....	18
Hình 2.5: Cấu trúc câu điền khuyết.....	18
Hình 3.1: Mảng JSON flashcard.....	23
Hình 3.2: Mảng JSON MCQ.....	24
Hình 3.3: Mảng JSON fill.....	24
Hình 3.4: Mảng JSON tf.....	24
Hình 3.5: Sơ đồ use-case tổng quan mô tả các chức năng với tác nhân là người dùng.....	26
Hình 3.6: Lưu đồ hoạt động.....	35
Hình 3.7: Sơ đồ kiến trúc khối.....	36
Hình 3.8: Sơ đồ deployment.....	39
Hình 3.9: Sơ đồ dữ liệu.....	41
Hình 3.10: Cấu trúc thư mục và tổ chức mã nguồn.....	44
Hình 3.11: Ma trận truy vết yêu cầu.....	46
Hình 4.1: Kết quả chạy setup.bat.....	49
Hình 4.2: PDF mẫu thử nghiệm.....	53
Hình 4.3: So sánh đầu ra giữa Text ranking và AI.....	65
Hình 4.4: Trang chủ sau khi khởi động và truy cập http://localhost:5000	72
Hình 4.5: Trang chủ sau khi tải tệp lên thành công.....	73
Hình 4.6: Trang chủ khi tải tệp lên thành công và nhấn nút “Xử lý với AI”.....	74
Hình 4.7: Tab Tóm tắt.....	74
Hình 4.8: Tab Mindmap - sơ đồ tư duy.....	75
Hình 4.9: Tab Flashcard - thẻ ghi nhớ.....	76
Hình 4.10: Tab Quiz - bộ câu hỏi.....	77
Hình 4.11: Lưu vào CSDL.....	78
Hình 4.12: Tệp dữ liệu JSON dùng để nhập/xuất dữ liệu.....	78
Hình 5.1: So sánh học tập thủ công và học cùng ứng dụng AI.....	83

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Tiếng Anh	Tiếng Việt
1	AI	Artificial Intelligence	Trí tuệ nhân tạo
2	LLM	Large Language Model	Mô hình ngôn ngữ lớn
3	API	Application Programming Interface	Giao diện lập trình ứng dụng
4	REST API	Representational State Transfer Application Programming Interface	API theo phong cách REST (truyền trạng thái đại diện)
5	HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
6	HTTPS	HyperText Transfer Protocol Secure	Giao thức truyền tải siêu văn bản bảo mật
7	JSON	JavaScript Object Notation	Định dạng dữ liệu kiểu đối tượng JavaScript
8	HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
9	CSS	Cascading Style Sheets	Ngôn ngữ định kiểu cho trang web
10	JS	JavaScript	Ngôn ngữ lập trình JavaScript (phía trình duyệt)
11	UI	User Interface	Giao diện người dùng
12	UX	User Experience	Trải nghiệm người dùng
13	MVC	Model – View – Controller	Mô hình Model – View – Controller
14	PDF	Portable Document Format	Định dạng tài liệu di động (tệp .pdf)
15	TXT	Text (file)	Tệp văn bản thuần (tệp .txt)
16	MCQ	Multiple Choice Question	Câu hỏi trắc nghiệm nhiều lựa chọn
17	DBMS	Database Management System	Hệ quản trị cơ sở dữ liệu
18	CDN	Content Delivery Network	Mạng phân phối nội dung

MỞ ĐẦU

Trong những năm gần đây, khối lượng tài liệu học tập dưới dạng số (PDF, slide, giáo trình điện tử...) tăng lên rất nhanh. Sinh viên thường phải đọc hàng chục, thậm chí hàng trăm trang tài liệu cho một học phần, trong khi quỹ thời gian ôn tập có hạn. Việc tự tóm tắt nội dung, hệ thống hóa kiến thức thành sơ đồ tư duy, xây dựng bộ câu hỏi luyện tập hay thẻ ghi nhớ (flashcard) thường đòi hỏi nhiều công sức lặp lại, dễ gây quá tải và thiếu nhất quán trong quá trình tự học. Song song đó, sự phát triển mạnh mẽ của các mô hình ngôn ngữ lớn (Large Language Models) và các dịch vụ AI đã mở ra khả năng tự động hoá nhiều tác vụ xử lý văn bản, hỗ trợ người học tập trung hơn vào việc hiểu và vận dụng kiến thức.

Xuất phát từ bối cảnh đó, đề án môn học “*Xây dựng ứng dụng web hỗ trợ học tập chuyển đổi tài liệu thành bộ câu hỏi, thẻ ghi nhớ và sơ đồ tư duy*” được thực hiện với mục tiêu xây dựng một trợ lý học tập chạy trên trình duyệt, cho phép người dùng tải lên tài liệu PDF/TXT và nhận lại các nội dung học tập đã được AI xử lý. Cụ thể, hệ thống sử dụng dịch vụ OpenAI (mô hình GPT-4.1-mini) để phân tích nội dung tài liệu và sinh ra bốn dạng kết quả chính: (i) bản tóm tắt kiến thức cốt lõi, (ii) sơ đồ tư duy ở dạng code Mermaid, (iii) bộ thẻ ghi nhớ hỏi–đáp (flashcard) và (iv) bộ câu hỏi quiz gồm câu hỏi trắc nghiệm, đúng/sai và điền khuyết có thể chấm điểm trực tiếp trên web. Toàn bộ quá trình được trình bày thông qua một giao diện web hiện đại, sáng, thân thiện, sử dụng tiếng Việt, giúp người học dễ dàng thao tác mà không cần cài đặt thêm ứng dụng.

Về mặt kỹ thuật, đề án triển khai một kiến trúc ứng dụng web gồm backend viết bằng Python với framework Flask và frontend thuần HTML/CSS/JavaScript. Ở phía server, chương trình đảm nhiệm việc nhận tệp người dùng tải lên, trích xuất văn bản từ tệp PDF bằng thư viện PyPDF2 hoặc đọc trực tiếp các file text đơn giản, sau đó xây dựng prompt và gọi OpenAI API thông qua thư viện openai. Kết quả trả về có cấu trúc JSON gồm bốn trường chính: summary, mindmap_mermaid,

flashcards, quizzes. Ở phía client, tệp static/index.html, static/styles.css và static/script.js tổ chức giao diện theo dạng các tab chức năng (Trang chủ, Tóm tắt, Sơ đồ tư duy, Flashcard, Quiz), cập nhật trạng thái tải tệp, hiển thị tiến trình “Xử lý với AI”, đồng bộ dữ liệu trả về và hỗ trợ các tương tác học tập như lật thẻ, chuyển thẻ trước/sau, làm bài và xem kết quả quiz.

Về mặt chức năng, ứng dụng tập trung giải quyết bài toán “biến một tài liệu thô thành một bộ nội dung phục vụ học tập” theo một luồng liên mạch: người dùng tải tệp PDF/TXT → hệ thống trích xuất nội dung và gửi lên OpenAI → mô hình sinh ra tóm tắt, mindmap, flashcard, quiz → kết quả được trình bày trên các tab riêng biệt để người học vừa đọc nhanh phần tóm tắt, vừa có sơ đồ tư duy để hệ thống hóa, vừa có bộ flashcard và bài quiz để tự kiểm tra mức độ nắm vững kiến thức.

Trong phạm vi một đồ án môn học, đồ án tập trung vào việc phân tích yêu cầu của một ứng dụng hỗ trợ tự học dựa trên tài liệu số. Từ đó, đề xuất kiến trúc và luồng xử lý phù hợp với công nghệ web hiện đại và dịch vụ OpenAI. Tiếp theo, thiết kế và triển khai backend Flask, các hàm trích xuất văn bản, hàm sinh nội dung học tập từ OpenAI và cơ chế xử lý lỗi. Sau đó xây dựng giao diện web thân thiện gồm các tab Tóm tắt, Mindmap, Flashcard và Quiz, tối ưu cho trải nghiệm học tập từng bước. Cuối cùng cài đặt và chạy thử ứng dụng với các tài liệu mẫu để kiểm tra tính đúng đắn của luồng xử lý và khả năng sử dụng thực tế.

Cấu trúc báo cáo đồ án được tổ chức thành 5 chương chính:

- CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI
- CHƯƠNG 2. CƠ SỞ LÝ THUYẾT
- CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG
- CHƯƠNG 4. THỰC NGHIỆM
- CHƯƠNG 5. KẾT LUẬN

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

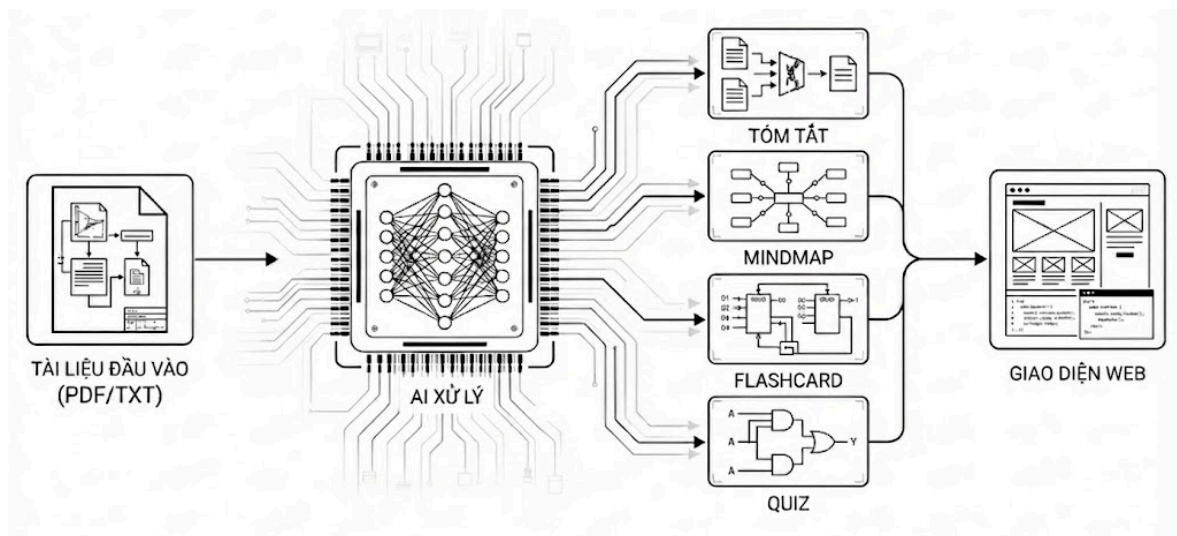
1.1. Giới thiệu chung

Trong bối cảnh chuyển đổi số mạnh mẽ hiện nay, phần lớn tài liệu học tập của sinh viên được phân phối dưới dạng tệp số như PDF, TXT, slide,... Khối lượng tài liệu lớn, độ dài nhiều trang và sự đa dạng về cấu trúc (giáo trình, bài báo, đề cương, ghi chú...) khiến việc tự đọc, tự tóm tắt và tự xây dựng hệ thống câu hỏi ôn tập trở thành một nhiệm vụ tốn nhiều thời gian và dễ gây quá tải.

Xuất phát từ thực tế trên, đề án được thực hiện với mục tiêu xây dựng một ứng dụng web hỗ trợ học tập, cho phép người dùng tải lên tài liệu dạng PDF/TXT, sau đó tự động chuyển đổi tài liệu đó thành:

- Bản tóm tắt nội dung giúp nắm nhanh ý chính.
- Sơ đồ tư duy (mindmap) dưới dạng code Mermaid để có thể trực quan hóa cấu trúc kiến thức.
- Bộ thẻ ghi nhớ (flashcard) hỏi – đáp, có đánh dấu mức độ khó.
- Bộ câu hỏi quiz gồm câu trắc nghiệm, đúng/sai và câu điền khuyết. Có thể làm trực tiếp trên web và chấm điểm tự động.

Ứng dụng được triển khai theo mô hình web client–server, với backend viết bằng Python/Flask và frontend bằng HTML/CSS/JavaScript thuần. Việc phân tích nội dung tài liệu và sinh ra tóm tắt, mindmap, flashcard, quiz được đảm nhiệm bởi mô hình GPT-4.1-mini thông qua API, còn phần trình bày kết quả và tương tác học tập được triển khai trên trình duyệt.



Hình 1.1: Sơ đồ quy trình xử lý nội dung học tập bằng AI.

1.2. Lý do chọn đề tài

Trong thực tế học tập, sinh viên thường gặp một số khó khăn điển hình:

- **Tốn nhiều thời gian tóm tắt:**

Mỗi học phần có thể kèm theo hàng chục đến hàng trăm trang tài liệu. Nếu tự tóm tắt thủ công, sinh viên phải đọc – lọc thông tin – ghi chú lại, để bỏ sót ý chính hoặc lặp lại nội dung.

- **Thiếu hệ thống câu hỏi luyện tập chất lượng:**

Không phải môn học nào cũng có sẵn ngân hàng câu hỏi. Nhiều sinh viên muốn tự tạo câu hỏi để luyện tập nhưng thiếu thời gian và phương pháp.

- **Chưa tận dụng tốt các dạng biểu diễn kiến thức khác nhau:**

Mindmap, flashcard, quiz là những hình thức đã được chứng minh hữu ích cho việc ghi nhớ và ôn tập, nhưng việc tự chuẩn bị chúng từ tài liệu thô không hề đơn giản.

- **Chưa khai thác hiệu quả các công cụ AI sẵn có:**

Dù các API AI đã phổ biến, việc tích hợp chúng vào một ứng dụng

web cụ thể, tối ưu cho nghiệp vụ học tập và thân thiện với người dùng cuối, vẫn là một khoảng trống mà sinh viên có thể khai thác.

Đề tài “Xây dựng ứng dụng web hỗ trợ học tập chuyển đổi tài liệu thành bộ câu hỏi, thẻ ghi nhớ và sơ đồ tư duy” được lựa chọn nhằm:

- Giải quyết trực tiếp một nhu cầu thực tế của sinh viên, giáo viên, người tự học.
- Tận dụng được sức mạnh của AI/LLM nhưng vẫn yêu cầu người thực hiện nắm vững kiến thức nền về lập trình web, kiến trúc hệ thống, thiết kế giao diện, xử lý file và làm việc với API.
- Đáp ứng định hướng của môn học về phát triển ứng dụng web tích hợp các kỹ thuật khai thác dữ liệu, khuyến nghị, Machine Learning trong các phiên bản phát triển tiếp theo.

1.3. Mục tiêu của đề tài

1.3.1. Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là xây dựng được một ứng dụng web hoàn chỉnh đóng vai trò như một trợ lý học tập: nhận tài liệu đầu vào là tệp PDF/TXT và sinh ra các cấu trúc nội dung phục vụ học tập (tóm tắt, mindmap, flashcard, quiz) một cách tự động, trực quan và dễ sử dụng trên trình duyệt.

1.3.2. Mục tiêu cụ thể

Từ mục tiêu tổng quát, đề tài đặt ra các mục tiêu cụ thể sau:

Mục tiêu về chức năng:

- Xây dựng chức năng upload tài liệu: hỗ trợ ít nhất hai định dạng chính là PDF và TXT.

- Xây dựng giao diện web với các tab chức năng: Trang chủ, Tóm tắt, Mindmap, Flashcard, Quiz.
- Xây dựng cơ sở dữ liệu để lưu lại kết quả xử lý để sử dụng lại sau, không cần gọi AI nhiều lần. Cùng chức năng Nhập/Xuất dữ liệu JSON để lưu trữ và chia sẻ.

Mục tiêu về chất lượng và trải nghiệm người dùng:

- Giao diện sáng, hiện đại, dùng font Inter, màu sắc nhẹ nhàng, có card, shadow, badge, trạng thái hover/focus rõ ràng.
- Có đầy đủ thông báo trạng thái trong quá trình xử lý:
 - Chưa tải tệp lên, đang tải tệp, tải tệp thành công.
 - Đang xử lý với AI, xử lý xong hoặc có lỗi.

Mục tiêu về kỹ thuật:

- Tuân thủ mô hình client–server, trao đổi dữ liệu qua JSON.
- Sử dụng Python 3.x, Flask, openai, PyPDF2, python-dotenv,...

1.4. Đối tượng phục vụ, phạm vi và giới hạn của đề tài

1.4.1. Đối tượng phục vụ

Ứng dụng web hướng đến các nhóm người dùng chính:

- **Sinh viên:** cần tóm tắt nhanh các chương giáo trình, tài liệu bài giảng để ôn thi.
- **Giáo viên, trợ giảng:** muốn nhanh chóng có bộ câu hỏi tham khảo hoặc flashcard để tổ chức hoạt động trên lớp.

- **Người tự học:** thường xuyên đọc ebook, tài liệu PDF/TXT và muốn có công cụ hỗ trợ hệ thống hóa kiến thức.

1.4.2. Phạm vi chức năng

Trong phạm vi của đề án, hệ thống tập trung vào xử lý một tệp tài liệu tại một thời điểm, với các nhóm chức năng chính:

- **Nhập tài liệu:** chọn tệp hoặc kéo thả, hiển thị tên tệp và trạng thái.
- **Xử lý với AI:** gửi văn bản lên OpenAI, sinh tóm tắt + mindmap + flashcard + quiz theo schema cố định.
- **Trình bày kết quả:** chia thành các tab riêng biệt, cho phép học/tương tác ngay trên trình duyệt.
- **Cơ sở dữ liệu:** Lưu trữ các kết quả xử lý.
- **Nhập/Xuất JSON:** lưu, chia sẻ và đọc lại kết quả xử lý.

Hệ thống **không triển khai** (trong phiên bản hiện tại):

- Cơ chế đăng nhập, phân quyền, lưu tài khoản người dùng.
- Các thuật toán gợi ý, cá nhân hóa nội dung dựa trên lịch sử học tập (mới dừng ở mức định hướng phát triển).

Những chức năng trên được xác định là phạm vi mở rộng trong tương lai.

1.5. Phương pháp thực hiện

Để đạt được các mục tiêu đã nêu, đề án được triển khai theo các bước chính sau:

(1) Nghiên cứu tài liệu và xác định yêu cầu

- Tìm hiểu nhu cầu thực tế của người học khi làm việc với tài liệu PDF/TXT: tóm tắt, hệ thống hóa, tạo câu hỏi luyện tập, flashcard.
- Khảo sát tài liệu hướng dẫn của OpenAI, Flask, PyPDF2, Mermaid để xác định khả năng tích hợp.
- Xác định rõ các nhóm chức năng và luồng nghiệp vụ của ứng dụng: upload → xử lý với AI → hiển thị kết quả → lưu trữ và nhập/xuất dữ liệu.

(2) Đề xuất kiến trúc và thiết kế giao diện

- Lựa chọn mô hình web một trang (single page) đơn giản với nhiều tab, mỗi tab tập trung vào một dạng nội dung.
- Đề xuất kiến trúc backend gồm các thành phần: đọc file, trích xuất văn bản, chuẩn bị prompt, gọi OpenAI, xử lý JSON.
- Thiết kế giao diện bằng HTML/CSS với các thành phần card, tab, badge, nút bấm, trạng thái để đảm bảo trải nghiệm sử dụng mạch lạc.

(3) Cài đặt backend Flask

- Hiện thực file app.py:
 - Cấu hình Flask và static folder (static/).
 - Khai báo biến môi trường, khởi tạo OpenAI client.
 - Cài đặt SYSTEM_PROMPT và JSON schema cho kết quả.
 - Cài đặt hàm `extract_text_from_file()` để trích xuất văn bản từ PDF/TXT.
 - Cài đặt hàm `generate_learning_materials()` để giới hạn độ dài text, gửi request và parse JSON.

- Cài đặt endpoint `/api/process_document` với xử lý lỗi rõ ràng (thiếu file, lỗi đọc file, lỗi JSON, lỗi API).

(4) Cài đặt frontend HTML/CSS/JS

- Xây dựng `static/index.html` với cấu trúc: header, thanh tab, các section `tab-home`, `tab-summary`, `tab-mindmap`, `tab-flashcards`, `tab-quiz`, khối nhập/xuất JSON, footer.
- Xây dựng `static/styles.css` với hệ thống biến màu, card, layout, trạng thái... để giao diện sáng, hiện đại.
- Xây dựng `static/script.js`:
 - Quản lý trạng thái ứng dụng (`appState`).
 - Xử lý upload/kéo thả file, cập nhật trạng thái, gọi API bằng `fetch`.
 - Render tóm tắt, render mindmap bằng Mermaid, cơ chế flashcard trung tâm và grid, quiz và chấm điểm, nhập/xuất JSON.

(5) Thử nghiệm và hoàn thiện

- Chạy ứng dụng ở chế độ development (`localhost:5000`).
- Thử nghiệm với một số tài liệu mẫu (PDF/TXT) để kiểm tra:
 - Khả năng trích xuất văn bản.
 - Khả năng sinh đúng format JSON của mô hình.
 - Hiển thị đúng ở mỗi tab và tính năng tương tác (flip flashcard, chấm quiz, copy Mermaid,...).

1.6. Tổng quan công nghệ, kiến trúc và tổ chức mã nguồn

Đồ án sử dụng các công nghệ chính sau:

- **Ngôn ngữ & Framework backend:**

- Python 3.x.
- Flask – framework nhẹ cho ứng dụng web.

- **Thư viện backend:**

- openai – gọi OpenAI Chat Completions API (mô hình GPT-4.1-mini).
- PyPDF2 – đọc nội dung file PDF.
- python-dotenv – load biến môi trường từ file .env.

- **Frontend:**

- HTML5, CSS3, JavaScript thuần (không sử dụng framework frontend).
- Thư viện Mermaid (qua CDN) để hiển thị sơ đồ tư duy từ code Mermaid.
- Google Fonts (Inter) để đồng bộ kiểu chữ.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Nền tảng kiến trúc ứng dụng web hỗ trợ học tập

Ứng dụng web trong đồ án được xây dựng theo mô hình web client–server, HTTP và JSON đơn giản:

Trong mô hình client–server, trình duyệt đóng vai trò client, gửi yêu cầu HTTP (thường là POST) lên server Flask tại endpoint `/api/process_document`, kèm theo file tải liệu. Server xử lý, gọi OpenAI và trả về kết quả ở dạng JSON. Cách tổ chức này tương tự một REST API đơn giản, thuận tiện cho việc mở rộng hoặc tích hợp về sau.

Dữ liệu trao đổi giữa frontend – backend – OpenAI đều sử dụng JSON (JavaScript Object Notation) vì:

- Dễ biểu diễn các cấu trúc lồng nhau (tóm tắt, mindmap, flashcards, quizzes).
- Tương thích tốt với JavaScript trên trình duyệt và các thư viện Python (`json.loads`, `json.dumps`).
- Hỗ trợ kiểm tra, xử lý lỗi tương đối tiện lợi.

2.2. Cơ sở lý thuyết về mô hình ngôn ngữ lớn và OpenAI Chat Completions

2.2.1. Tổng quan mô hình ngôn ngữ lớn (LLM)

Mô hình ngôn ngữ lớn (Large Language Model – LLM) là các mô hình học sâu được huấn luyện trên khối lượng dữ liệu văn bản rất lớn, có khả năng:

- Hiểu ngữ cảnh, sinh văn bản tự nhiên, trả lời câu hỏi, tóm tắt, dịch, tạo cấu trúc dữ liệu từ văn bản,...
- Xử lý được nhiều ngôn ngữ, trong đó có tiếng Việt.

Trong đồ án, mô hình được sử dụng là `gpt-4.1-mini`, thông qua thư viện `openai` và hàm `client.chat.completions.create(...)`. Với chế độ Chat Completions, nghĩa là:

- Mỗi yêu cầu gồm một danh sách messages:

- system: mô tả vai trò/định hướng cho mô hình.
- user: nội dung yêu cầu cụ thể (prompt) cùng tài liệu đầu vào.
- Kết quả trả về gồm nhiều choices, mỗi choice có message.content là đoạn văn bản do mô hình sinh.

Đề án tận dụng khả năng của LLM để:

- Phân tích nội dung tài liệu học tập.
- Trích rút các ý chính để tóm tắt.
- Đề xuất cấu trúc sơ đồ tư duy dạng code Mermaid.
- Sinh flashcard hỏi – đáp có gán độ khó.
- Sinh bộ câu hỏi quiz đa dạng (trắc nghiệm + đúng/sai + điền khuyết).

2.2.2. Thiết kế prompt và JSON schema cho nội dung học tập

Để đảm bảo mô hình trả về dữ liệu đúng định dạng, app.py định nghĩa một biến SYSTEM_PROMPT khá chi tiết. Nội dung chính của prompt:

- **Xác định vai trò:**
“Bạn là trợ lý học tập AI. Nhiệm vụ của bạn là chuyển một tài liệu học tập thành: tóm tắt, sơ đồ tư duy, bộ flashcard, bộ câu hỏi quiz...”
- **Yêu cầu trả về** duy nhất một JSON hợp lệ, không có Markdown, không có
- **Mô tả rõ schema:**

```

{
  "summary": "chuỗi văn bản, có thể gồm nhiều đoạn hoặc bullet, tóm tắt những ý chính",
  "mindmap_mermaid": "chuỗi code Mermaid, kiểu mindmap hoặc flowchart, nút trung tâm là chủ đề chính",
  "flashcards": [
    {
      "front": "câu hỏi/ngắn gọn, 1 khái niệm cần nhớ",
      "back": "câu trả lời/nghĩa đầy đủ, súc tích",
      "difficulty": "easy" hoặc "medium" hoặc "hard"
    }
  ],
  "quizzes": [
    {
      "type": "mcq",
      "question": "câu hỏi trắc nghiệm",
      "options": ["A. ...", "B. ...", "C. ...", "D. ..."],
      "answer": "A" // chỉ là ký tự A/B/C/D
    },
    {
      "type": "fill",
      "question": "câu có chỗ trống ____ cần điền",
      "answer": "đáp án đúng"
    },
    {
      "type": "tf",
      "question": "phát biểu đúng hay sai",
      "answer": true
    }
  ]
}

```

Hình 2.1: Mô tả schema.

- **Yêu cầu về số lượng:**

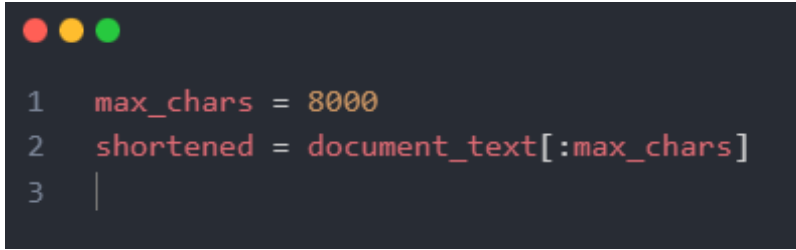
- 8–15 flashcard.
- 8–12 câu hỏi quiz, khoảng 60% trắc nghiệm, 20% đúng/sai và 20% điền khuyết.

Như vậy, ở cấp độ lý thuyết, hệ thống áp dụng prompt engineering và schema-based generation để kiểm soát định dạng đầu ra của LLM, giúp frontend có thể xử lý và hiển thị nội dung một cách an toàn và nhất quán.

2.2.3. Quản lý ngữ cảnh và giới hạn độ dài văn bản

Do LLM có giới hạn về số lượng token đầu vào, đồ án áp dụng chiến lược đơn giản nhưng hiệu quả:

- Chỉ gửi lên mô hình tối đa 8.000 ký tự của tài liệu:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Python and consists of three lines:

```
1 max_chars = 8000
2 shortened = document_text[:max_chars]
3 |
```

Hình 2.2: Giản lược ký tự.

Điều này đảm bảo:

- Tránh lỗi do vượt quá giới hạn ngữ cảnh của mô hình.
- Giữ thời gian phản hồi ở mức chấp nhận được.

Trong tương lai, có thể mở rộng bằng các kỹ thuật như:

- Chia tài liệu lớn thành nhiều đoạn (chunking) và tóm tắt từng phần.
- Áp dụng tóm tắt phân cấp (hierarchical summarization).

Nhưng trong phạm vi đồ án hiện tại, phương pháp cắt theo ký tự là phù hợp với yêu cầu và cấu trúc mã nguồn đã triển khai.

2.3. Nền tảng dữ liệu văn bản và trích xuất từ tệp PDF/TXT

Tài liệu đầu vào được giả định chủ yếu là:

- PDF: giáo trình, slide xuất ra PDF, bài báo, tài liệu học tập.
- TXT / file text đơn giản: ghi chú, nội dung trích xuất sẵn.

Đặc điểm chung:

- Nội dung mang tính liên tục, có cấu trúc logic (chương, mục, tiêu đề, đoạn văn...).
- Có thể chứa tiếng Việt có dấu, yêu cầu xử lý đúng mã hóa ký tự.
- Với PDF, việc trích xuất text phụ thuộc vào cách tài liệu được tạo (PDF từ scan, PDF text-based, có nhiều cột, bảng,...).

Từ đó, có thể sử dụng PyPDF2 - một thư viện Python mã nguồn mở hỗ trợ làm việc với tệp PDF ở mức văn bản, cho phép:

- Mở và đọc cấu trúc tài liệu PDF.
- Truy cập từng trang (page) của tài liệu.
- Trích xuất chuỗi văn bản từ mỗi trang (đối với PDF dạng text-based).

Trong đồ án, PyPDF2 được sử dụng để trích xuất văn bản thô từ các tệp PDF mà người dùng tải lên, làm đầu vào cho mô hình ngôn ngữ lớn. Quy trình sử dụng có thể tóm tắt như sau:

- Backend nhận file PDF từ phía client (thông qua Flask).
- Tạo đối tượng PdfReader từ file này.
- Duyệt qua từng trang và gọi phương thức `extract_text()` để lấy nội dung text.
- Ghép các đoạn text lại thành một chuỗi hoàn chỉnh, sau đó truyền vào hàm sinh nội dung học tập.

Như vậy, về mặt cơ sở lý thuyết, PyPDF2 đóng vai trò là “cầu nối” giữa tệp PDF và dạng văn bản thuần (plain text) mà mô hình ngôn ngữ lớn có thể xử lý.

2.4. Cơ sở lý thuyết về các dạng nội dung học tập sinh ra từ tài liệu

Ứng dụng sinh ra bốn dạng nội dung chính, tương ứng với bốn tab trên giao diện: Tóm tắt, Mindmap, Flashcard, Quiz. Mỗi dạng đều có ý nghĩa lý thuyết trong hỗ trợ học tập.

2.4.1. Tóm tắt tài liệu

Tóm tắt (summarization) là quá trình rút gọn văn bản gốc thành một phiên bản ngắn hơn nhưng vẫn giữ được những ý chính. Trong đồ án:

- Trường summary là một chuỗi văn bản có thể gồm nhiều đoạn hoặc bullet.

Mục tiêu:

- Giúp người học nắm nhanh cấu trúc và nội dung chính của tài liệu.
- Hỗ trợ định hướng trước khi đọc chi tiết.

2.4.2. Sơ đồ tư duy (mindmap) và Mermaid

Sơ đồ tư duy (mindmap) là một kỹ thuật trực quan hóa kiến thức, tổ chức thông tin theo dạng cây, với:

- Nút trung tâm là chủ đề chính.
- Các nhánh cấp 1 thể hiện các nhóm nội dung lớn.
- Các nhánh nhỏ hơn thể hiện chi tiết, ví dụ, lưu ý.

Trong đồ án:

- Trường mindmap_mermaid là chuỗi code theo cú pháp Mermaid.

Mermaid là thư viện JavaScript cho phép mô tả sơ đồ bằng văn bản (text-based diagram). Việc dùng Mermaid giúp:

- Không cần tự vẽ sơ đồ bằng tay trên frontend.

- Người dùng có thể copy code để chỉnh sửa thêm trên Mermaid Live Editor.

2.4.3. Flashcard hỏi – đáp

Flashcard là một kỹ thuật học thuộc phổ biến, dựa trên hai mặt “hỏi – đáp”:

- Mặt trước (front): gợi ý, câu hỏi, khái niệm.
- Mặt sau (back): câu trả lời hoặc định nghĩa đầy đủ.

Trong schema JSON của đề án, mỗi flashcard có cấu trúc:

```
"flashcards": [  
  {  
    "front": "câu hỏi/ngắn gọn, 1 khái niệm cần nhớ",  
    "back": "câu trả lời/nghĩa đầy đủ, súc tích",  
    "difficulty": "easy" hoặc "medium" hoặc "hard"  
  }  
],
```

Hình 2.3: Cấu trúc schema của flashcard.

Mô hình được yêu cầu tạo 8–15 flashcard, với trường difficulty phân loại mức độ khó.

Về mặt lý thuyết, flashcard giúp:

- Tăng khả năng nhớ lâu dựa trên cơ chế lặp lại truy hồi (retrieval practice).
- Cho phép người học chủ động tự kiểm tra thay vì chỉ đọc.

2.4.4. Quiz trắc nghiệm, đúng/sai và điền khuyết

Quiz là dạng bài tập giúp người học tự đánh giá mức độ hiểu tài liệu. Đề án sử dụng ba loại quiz:

- Câu trắc nghiệm (MCQ):

```
{
  "type": "mcq",
  "question": "câu hỏi trắc nghiệm",
  "options": ["A. ...", "B. ...", "C. ...", "D. ..."],
  "answer": "A" // chỉ là ký tự A/B/C/D
},
```

Hình 2.4: Cấu trúc câu trắc nghiệm.

- Câu điền khuyết (fill):

```
{
  "type": "fill",
  "question": "câu có chỗ trống ____ cần điền",
  "answer": "đáp án đúng"
},
```

Hình 2.5: Cấu trúc câu điền khuyết.

- Câu đúng sai (tf):

```
{
  "type": "tf",
  "question": "phát biểu đúng hay sai",
  "answer": true
}
```

Mô hình được yêu cầu sinh 8–12 câu quiz, trong đó khoảng 60% MCQ, 20% fill, 20% tf.

2.5. Các thư viện chính và cơ chế xử lý lỗi, bảo mật

Để xây dựng backend và kết nối tới dịch vụ AI, đồ án sử dụng ba thư viện chính: Flask, openai và python-dotenv, đồng thời áp dụng một số cơ chế xử lý lỗi và bảo mật cơ bản.

- **Flask** là micro framework Python dùng để xây dựng ứng dụng web và API. Trong đề tài, Flask khởi tạo server backend, định nghĩa endpoint `/api/process_document` để nhận file người dùng upload và trả kết quả đã xử lý cho frontend dưới dạng JSON. Flask có cấu trúc gọn, dễ tích hợp với các thư viện khác (PyPDF2, openai...) và thuận tiện chạy trên môi trường localhost.
- **Thư viện openai** là SDK giúp gọi OpenAI API từ Python. Backend khởi tạo client với khoá API lấy từ biến môi trường, gửi yêu cầu Chat Completions tới mô hình gpt-4.1-mini với các messages (system, user), sau đó nhận lại nội dung mô hình sinh và parse về đúng JSON schema (summary, mindmap_mermaid, flashcards, quizzes). Việc dùng SDK giúp giảm thao tác thủ công với HTTP, hạn chế lỗi và tập trung hơn vào logic xử lý nội dung học tập.
- **python-dotenv** hỗ trợ đọc biến môi trường từ file `.env`, đặc biệt là `OPENAI_API_KEY`. Cách này giúp tách API key và các thông tin nhạy cảm ra khỏi mã nguồn, tránh ghi cứng trong file `.py` và hạn chế nguy cơ lộ khoá khi chia sẻ project hoặc đưa lên kho lưu trữ công cộng.

Về xử lý lỗi, backend bao bọc các thao tác chính (nhận file, đọc PDF, gọi OpenAI, parse JSON) trong các khối try/except. Khi xảy ra lỗi như: không có file, file sai định dạng, không trích xuất được văn bản, thiếu `OPENAI_API_KEY`, lỗi mạng hoặc mô hình trả về JSON không hợp lệ..., server sẽ bắt ngoại lệ, sinh một response JSON mô tả lỗi (ví dụ `{"success": false, "error": "Không đọc được nội dung PDF"}`) và trả cho frontend để hiển thị thông báo rõ ràng. Nhờ vậy, ứng dụng không bị “crash” khi gặp dữ liệu bất thường và trải nghiệm người dùng vẫn mạch lạc.

Về bảo mật cơ bản, báo cáo nhấn mạnh một số nguyên tắc:

- Không ghi cứng khoá API trong mã nguồn, mà luôn đọc từ biến môi trường (qua python-dotenv).
- Chỉ xử lý các định dạng file được hỗ trợ (PDF, TXT) và không thực thi bất kỳ nội dung nào từ file người dùng, chỉ đọc dưới dạng dữ liệu văn bản.
- Thông báo lỗi ở mức nghiệp vụ, tránh hiển thị chi tiết nội bộ hoặc stack trace trên giao diện.

Trong phạm vi một đồ án môn học, những thực hành trên là phù hợp: vừa đảm bảo ứng dụng hoạt động ổn định, vừa thể hiện ý thức về an toàn khi làm việc với file người dùng và khoá API.

2.6. Tóm tắt chương

Trong Chương 2, báo cáo đã trình bày các cơ sở lý thuyết và nền tảng công nghệ làm nền cho đề tài:

- Kiến trúc web client–server với frontend HTML/CSS/JS và backend Flask, sử dụng JSON làm định dạng trao đổi dữ liệu.
- Nguyên lý sử dụng mô hình ngôn ngữ lớn thông qua OpenAI Chat Completions API, kết hợp với thiết kế prompt và JSON schema để sinh nội dung học tập có cấu trúc.
- Kỹ thuật trích xuất văn bản từ PDF/TXT bằng PyPDF2 và các thao tác xử lý mã hóa ký tự.
- Cơ sở lý thuyết của bốn dạng nội dung được sinh ra: tóm tắt, mindmap (Mermaid), flashcard và quiz, cùng vai trò của chúng trong hỗ trợ học tập.
- Các thư viện chính (Flask, openai, PyPDF2, python-dotenv), cách tổ chức mã nguồn và những cơ chế xử lý lỗi, bảo mật căn bản.

Những nội dung này là nền tảng để sang Chương 3, báo cáo sẽ đi sâu vào phân tích yêu cầu và thiết kế chi tiết hệ thống, bám sát cấu trúc mã nguồn và các tính năng đã triển khai trong project.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu

3.1.1. Yêu cầu chức năng

Các yêu cầu chức năng chính của hệ thống có thể tổng hợp như sau:

Tải tài liệu học tập (Upload)

- Người dùng chọn hoặc kéo–thả một tệp tài liệu vào vùng upload trên tab Trang chủ.
- Hệ thống chấp nhận ít nhất các định dạng:
 - .pdf – được trích xuất nội dung bằng thư viện PyPDF2.
 - Các file text (ví dụ .txt) – được đọc trực tiếp dưới dạng chuỗi.
- Sau khi chọn file, giao diện hiển thị tên tệp và trạng thái “Đã chọn tệp...”.

Gửi tài liệu lên server và xử lý bằng AI

- Khi người dùng nhấn nút “Xử lý với AI”, frontend gọi API POST `/api/process_document` kèm theo file.
- Backend thực hiện:

- Trích xuất văn bản từ file (`extract_text_from_file`).
- Cắt bớt văn bản nếu quá dài (tối đa ~8.000 ký tự) để phù hợp giới hạn mô hình.
- Gọi OpenAI Chat Completions API (model `gpt-4.1-mini`) với `SYSTEM_PROMPT` và `user_prompt` đã thiết kế, yêu cầu trả về duy nhất một JSON hợp lệ theo schema.
- Parse nội dung JSON, đảm bảo các trường `summary`, `mindmap_mermaid`, `flashcards`, `quizzes` luôn tồn tại.
- Nếu xử lý thành công, backend trả về JSON để frontend cập nhật giao diện.

Sinh và hiển thị bản tóm tắt (Summary)

- Dữ liệu tóm tắt được lưu trong `appState.summary` (frontend).
- Tab “Tóm tắt” hiển thị nội dung tóm tắt theo dạng đoạn văn/bullet, tối ưu cho đọc lướt.
- Tab này ban đầu bị vô hiệu hóa (`tab-disabled`), chỉ được kích hoạt sau khi xử lý AI thành công.

Sinh và hiển thị sơ đồ tư duy (Mindmap)

- Dữ liệu sơ đồ tư duy là code Mermaid, lưu trong `appState.mindmap_mermaid`.
- Tab “Mindmap” sử dụng thư viện Mermaid (từ CDN) để:
 - Render sơ đồ ra SVG trong `<div id="mindmapDiagramWrapper">`.

- Hiển thị code Mermaid trong `<pre id="mindmapCode">` để người dùng có thể copy.
- Có nút “Copy code Mermaid” cho phép copy toàn bộ code vào clipboard.

Sinh và hiển thị bộ flashcard

- Dữ liệu flashcard là một mảng đối tượng JSON có dạng:

```
"flashcards": [
  {
    "front": "câu hỏi/ngắn gọn, 1 khái niệm cần nhớ",
    "back": "câu trả lời/nghĩa đầy đủ, súc tích",
    "difficulty": "easy" hoặc "medium" hoặc "hard"
  }
],
```

Hình 3.1: Mảng JSON flashcard.

- Tab “Flashcard” cung cấp hai chế độ:

Thẻ trung tâm:

- Hiển thị một thẻ tại vị trí trung tâm (câu hỏi ở mặt trước, câu trả lời ở mặt sau).
- Nút < / > để chuyển thẻ trước/sau.
- Nút “Xem” để bật tắt câu trả lời.
- Badge mức độ khó (difficulty) với màu sắc khác nhau.
- Bộ đếm “i / N” thể hiện vị trí thẻ hiện tại.

- Xem tất cả thẻ:

- Nút “Xem tất cả thẻ” hiển thị grid chứa toàn bộ flashcard (front + back) để người dùng xem tổng quan.

Sinh và hiển thị bộ câu hỏi Quiz

Dữ liệu quiz là mảng JSON gồm ba loại câu hỏi:

- Trắc nghiệm (MCQ):

```
{  
  "type": "mcq",  
  "question": "câu hỏi trắc nghiệm",  
  "options": ["A. ...", "B. ...", "C. ...", "D. ..."],  
  "answer": "A" // chỉ là ký tự A/B/C/D  
},
```

Hình 3.2: Mảng JSON MCQ.

- Điền khuyết (fill):

```
{  
  "type": "fill",  
  "question": "câu có chỗ trống ____ cần điền",  
  "answer": "đáp án đúng"  
},
```

Hình 3.3: Mảng JSON fill.

- Đúng/Sai (tf):

```
{  
  "type": "tf",  
  "question": "phát biểu đúng hay sai",  
  "answer": true  
}
```

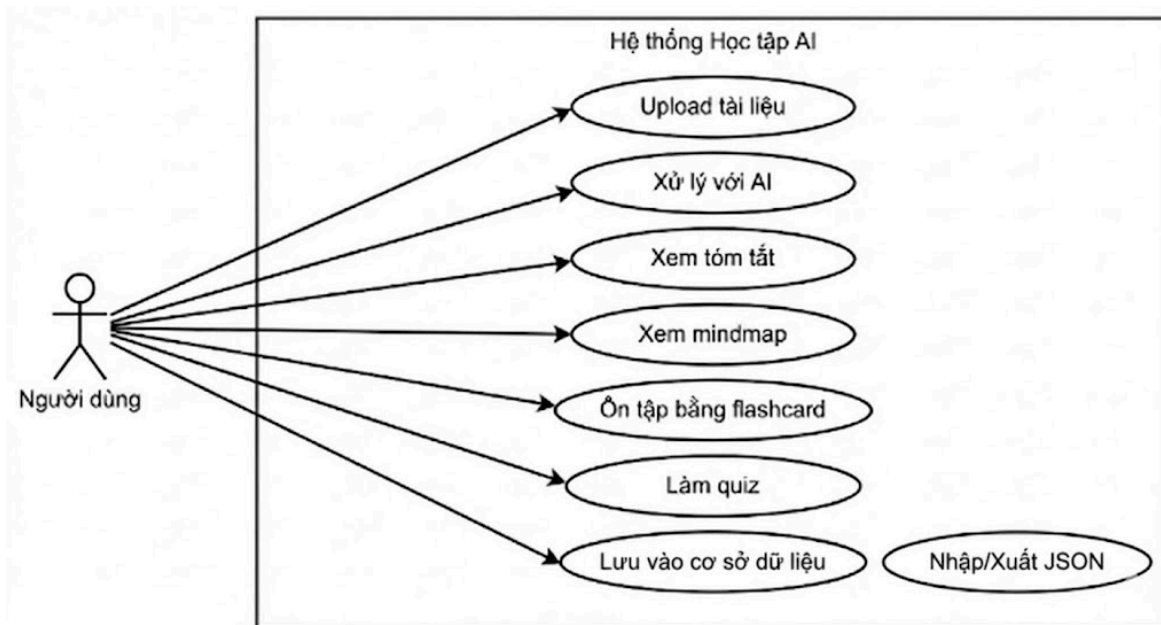
Hình 3.4: Mảng JSON tf.

- Tab “Quiz”:

- Render mỗi câu hỏi dưới dạng card: tiêu đề (Câu 1, Câu 2,...), badge loại câu (Trắc nghiệm/Điền khuyết), nội dung câu hỏi.
- Với câu MCQ + TF: hiển thị danh sách lựa chọn dạng radio.
- Với câu điền khuyết: hiển thị ô nhập text.
- Nút “Nộp bài”:
 - Duyệt tất cả câu hỏi, so sánh với answer.
 - Tính tổng số câu đúng, phần trăm.
 - Hiển thị kết quả tổng và feedback cho từng câu (đúng/sai, đáp án đúng).

Lưu trữ vào CSDL và Nhập/Xuất dữ liệu JSON

- Ở cuối giao diện có card “Nhập/Xuất dữ liệu”:
 - Nút “Xuất dữ liệu JSON”:
 - Lấy `appState` (`summary`, `mindmap_mermaid`, `flashcards`, `quizzes`, `source_file`).
 - Tải xuống file `.json` (tên gợi ý: `{ten_tai_lieu}_study_data.json`).
 - Nút “Nhập dữ liệu từ JSON”:
 - Người dùng chọn một file JSON do chính ứng dụng đã xuất.
 - Frontend parse JSON, cập nhật `appState`, render lại các tab kết quả mà không cần gọi AI lại.
 - Nút “Lưu vào CSDL”: Lưu các cấu trúc JSON vào CSDL.



Hình 3.5: Sơ đồ use-case tổng quan mô tả các chức năng với tác nhân là người dùng.

Dưới đây là bảng các yêu cầu chức năng:

Mã	Yêu cầu chức năng	Mô tả ngắn	Kết quả đầu ra
FR-01	Tải lên tài liệu	Người dùng chọn hoặc kéo thả tệp PDF/TXT	Tệp được chọn, sẵn sàng xử lý
FR-02	Trích xuất văn bản	Backend đọc nội dung từ PDF/TXT	Text thô (source_text)
FR-03	Xử lý AI tạo học liệu	Gọi LLM sinh summary/mindmap/flashcards/quizzes theo schema	JSON học liệu
FR-04	Hiển thị tóm tắt	Tab Summary hiển thị nội dung tóm tắt	Summary render trên UI
FR-05	Hiển thị mindmap Mermaid	Tab Mindmap render Mermaid + xem code	Sơ đồ + code Mermaid

FR-06	Ôn tập flashcard	Lật thẻ, xem đáp án, next/prev, xem tất cả	Trải nghiệm flashcard
FR-07	Làm bài quiz	Render quiz theo loại: MCQ, Điền khuyết, Đúng/Sai; chấm điểm	Kết quả đúng/sai, %
FR-08	Xuất dữ liệu ra JSON	Tải file JSON để lưu/ chia sẻ	File *_study_data.json
FR-09	Nhập dữ liệu từ JSON	Nạp lại học liệu từ file đã xuất	UI hiển thị dữ liệu nhập
FR-10	Lưu “bản ôn tập” vào CSDL	Người dùng đặt tên, lưu payload JSON vào SQLite	Bản ôn tập có id, thời gian
FR-11	Liệt kê bản ôn tập đã lưu	Combobox hiển thị danh sách (mới nhất trước)	Danh sách display_name
FR-12	Tải lại bản ôn tập từ CSDL	Chọn item trong combobox để nạp dữ liệu	UI cập nhật toàn bộ tab
FR-13	Tạo thêm Quiz	Sinh thêm quiz từ “ngữ cảnh hiện tại”, tránh trùng câu hỏi	Danh sách quiz được nổi thêm
FR-14	Tạo thêm Flashcard	Sinh thêm flashcard từ “ngữ cảnh hiện tại”, tránh trùng mặt trước	Danh sách flashcard nổi thêm
FR-15	Quản lý trạng thái nút/tab	Chỉ bật tab/nút khi có dữ liệu đủ điều kiện	Trải nghiệm mượt, ít lỗi

Bảng 3.1: Bảng liệt kê các yêu cầu chức năng.

3.1.2. Yêu cầu phi chức năng

Tính dễ sử dụng (Usability):

- Giao diện một trang (single page), chia theo tab, giúp người dùng nắm luồng thao tác rõ ràng. Home → Summary → Mindmap → Flashcard → Quiz.
- Có thông báo trạng thái rõ ràng: chưa có file, đang tải, đang xử lý, xử lý xong, lỗi. Giúp người dùng hiểu hệ thống đang làm gì.
- Ngôn ngữ hiển thị là tiếng Việt, phù hợp đối tượng sinh viên trong nước.

Hiệu năng và giới hạn tài nguyên:

- Văn bản gửi lên mô hình bị giới hạn độ dài (`max_chars = 8000`) nhằm:
 - Tránh vượt giới hạn token của mô hình.
 - Đảm bảo thời gian phản hồi ở mức chấp nhận được.
- Render Mermaid theo hướng “client-side” để hiển thị nhanh trên trình duyệt.
- Khi tạo thêm quiz/flashcard, giới hạn số phần tử đưa vào context (vd: flashcards lấy tối đa ~30, quiz lấy ~20...) để giảm độ dài prompt.

Độ tin cậy:

- Backend xử lý các lỗi phổ biến: thiếu file, tên file trống, không trích xuất được text, JSON model trả về sai định dạng.
- Frontend có `safeFetchJSON()` để chuẩn hóa xử lý lỗi HTTP, hiển thị thông báo thân thiện.

Tính bảo mật cơ bản:

- API key của OpenAI không hard-code trong mã nguồn, mà được lưu trong file `.env` dưới biến `OPENAI_API_KEY` và nạp qua `python-dotenv`.
- Server không ghi nội dung tài liệu lên đĩa; xử lý file trong bộ nhớ và trả kết quả trực tiếp cho client.

- Backend chỉ tiếp nhận đúng định dạng cần thiết (PDF/TXT theo mục tiêu).
- Không lưu thông tin nhạy cảm ngoài payload học liệu; mô hình lưu trữ tối giản.

Khả năng bảo trì và mở rộng:

- Mã nguồn tách rõ backend (app.py) và frontend (static/index.html, static/styles.css, static/script.js).
- Định dạng dữ liệu trao đổi qua lại được chuẩn hóa bằng một JSON schema rõ ràng (summary, mindmap_mermaid, flashcards, quizzes). Nhất quán giúp dễ bổ sung loại nội dung mới.
- Có thể thay đổi mô hình (ví dụ gpt-4.1-mini → model khác) bằng cách chỉnh sửa duy nhất ở generate_learning_materials().
- Các chức năng “tạo thêm” tách endpoint riêng, dễ mở rộng thêm chế độ “tạo đề theo độ khó”, “tạo theo chương”, ...

Tính di động (Portability)

- Chạy local trên máy cá nhân: Python + Flask + SQLite file-based.
- Có script setup.bat và run_server.bat hỗ trợ cài đặt và chạy nhanh.
- Frontend thuần HTML/CSS/JS, chạy trên mọi trình duyệt hiện đại.

3.2. Mô hình tác nhân, use-case và luồng nghiệp vụ

3.2.1. Tác nhân hệ thống

Hệ thống chỉ có một tác nhân chính là **Người dùng**:

- Là sinh viên, giáo viên hoặc người tự học.

- Tương tác qua trình duyệt, tải tài liệu, yêu cầu xử lý, đọc tóm tắt, xem mindmap, ôn tập flashcard, làm quiz, nhập/xuất JSON.

Ngoài ra còn có, **Tác nhân phụ:**

- Dịch vụ LLM (OpenAI-compatible API): nhận prompt và trả JSON học liệu.
- CSDL SQLite: lưu và cung cấp lại các “bản ôn tập”.

3.2.2. Các use-case chính

Từ các chức năng trong 3.1.1, ta có các use-case tiêu biểu:

UC1 – Upload tài liệu

- Tác nhân chọn tệp hoặc kéo–thả vào vùng upload.
- Hệ thống kiểm tra định dạng, hiển thị tên tệp, trạng thái “Đã chọn tệp...”.

UC2 – Xử lý tài liệu với AI

- Điều kiện tiên quyết: UC1 đã hoàn thành, có tệp hợp lệ.
- Tác nhân nhấn nút “Xử lý với AI”.
- Hệ thống gửi file lên backend, gọi OpenAI, lưu kết quả vào appState, mở khóa các tab Summary/Mindmap/Flashcard/Quiz.

UC3 – Xem tóm tắt

- Tác nhân chọn tab “Tóm tắt”.
- Hệ thống hiển thị nội dung summary.

UC4 – Xem sơ đồ tư duy

- Tác nhân chọn tab “Mindmap”.

- Hệ thống render mindmap từ mindmap_mermaid và cho phép copy code.

UC5 – Học bằng flashcard

- Tác nhân chọn tab “Flashcard”.
- Có thể:
 - Dùng Prev/Next để duyệt từng thẻ.
 - Bấm “Hiện/Ẩn đáp án” để tự kiểm tra.
 - Xem tất cả thẻ dưới dạng grid.

UC6 – Làm quiz

- Tác nhân chọn tab “Quiz”.
- Làm các câu trắc nghiệm và điền khuyết.
- Bấm “Nộp bài” để xem kết quả và feedback.

UC7 – Xuất dữ liệu JSON

- Tác nhân bấm “Xuất dữ liệu JSON”.
- Hệ thống tải xuống file .json chứa toàn bộ summary, mindmap_mermaid, flashcards, quizzes, source_file.

UC8 – Nhập dữ liệu từ JSON

- Tác nhân chọn file JSON đã export từ ứng dụng.
- Hệ thống đọc, parse JSON, cập nhật appState và render lại các tab kết quả.

UC9 – Lưu bản ôn tập vào CSDL

- Tác nhân nhấn “Lưu vào CSDL”.
- Hệ thống hỏi tên bản ôn tập.
- Hệ thống gửi JSON lên backend (name, source_file, data) và lưu vào SQLite.
- Hệ thống cập nhật lại combobox “Những bản ôn tập đã lưu” và tự chọn đúng bản vừa lưu.

UC10 – Tải bản ôn tập từ CSDL

- Điều kiện tiên quyết: Đã có ít nhất một bản ôn tập được lưu (UC9).
- Tác nhân chọn một mục trong combobox “Những bản ôn tập đã lưu”.
- Hệ thống gọi backend lấy dữ liệu session từ SQLite theo id.
- Hệ thống cập nhật appState bằng dữ liệu tải về và render lại các tab Summary/Mindmap/Flashcard/Quiz.

UC-11: Tạo thêm quiz

- Điều kiện tiên quyết: Có đủ ngữ cảnh và đã có danh sách quiz hiện tại để tránh trùng.
- Tác nhân nhấn “Tạo thêm Quiz” trong tab Quiz.
- Hệ thống build context (tài liệu + tóm tắt + quiz/flashcard hiện có) và gửi /api/generate_quiz kèm existing_quizzes.
- Backend gọi LLM theo prompt chuyên biệt và trả về danh sách quiz mới.
- Hệ thống nối quiz mới vào danh sách quiz hiện tại, cập nhật appState và render lại UI.

UC-12: Tạo thêm flashcard

- Điều kiện tiên quyết: Có đủ ngữ cảnh và đã có danh sách flashcard hiện tại để tránh trùng.
- Tác nhân nhấn “Tạo thêm Flashcard” trong tab Flashcard.
- Hệ thống build context và gửi `/api/generate_flashcards` kèm `existing_flashcards`.
- Backend gọi LLM theo prompt chuyên biệt và trả về danh sách flashcard mới.
- Hệ thống nối flashcard mới vào danh sách hiện tại, cập nhật `appState` và render lại UI.

3.2.3. Luồng hoạt động tổng quát

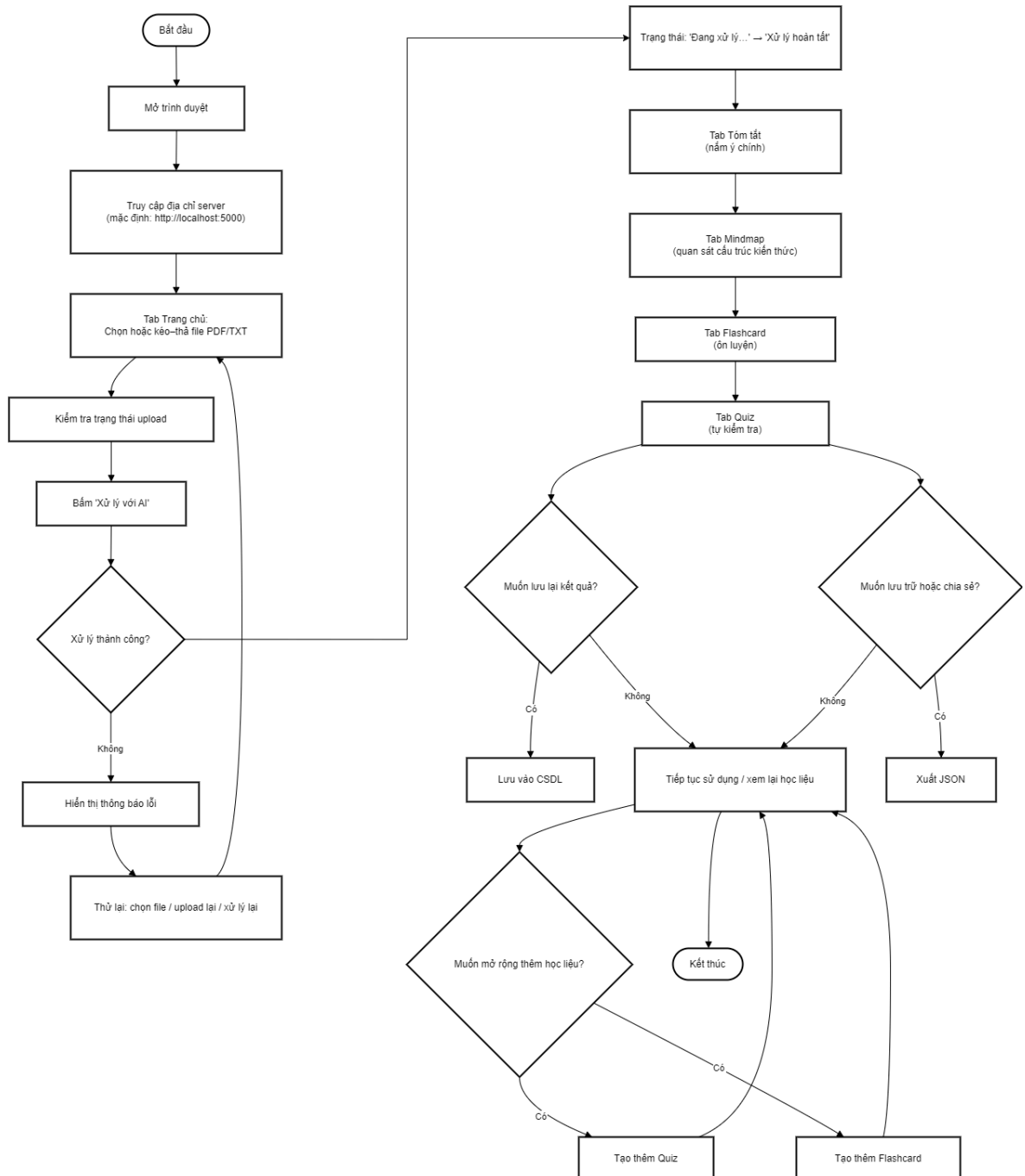
Luồng nghiệp vụ chuẩn khi sử dụng hệ thống:

1. Mở trình duyệt → truy cập địa chỉ server (mặc định: <http://localhost:5000>).
2. Tại tab Trang chủ: chọn hoặc kéo–thả file PDF/TXT.
3. Kiểm tra trạng thái upload → bấm “Xử lý với AI”.
4. Đợi trạng thái “Đang xử lý...” chuyển sang “Xử lý hoàn tất” (hoặc thông báo lỗi nếu có).
5. Lần lượt truy cập các tab:
 - a. Tóm tắt để nắm ý chính.
 - b. Mindmap để quan sát cấu trúc kiến thức.
 - c. Flashcard để ôn luyện.
 - d. Quiz để tự kiểm tra.

6. Nếu muốn lưu lại kết quả → Lưu vào CSDL.

7. Nếu muốn lưu trữ hoặc chia sẻ → Xuất JSON.

8. Có thể mở rộng thêm học liệu → Tạo thêm Quiz / Flashcard.



Hình 3.6: Lưu đồ hoạt động.

3.3. Thiết kế kiến trúc

3.3.1. Kiến trúc tổng thể 2 tầng

Hệ thống được thiết kế theo kiến trúc 2 tầng (client–server):

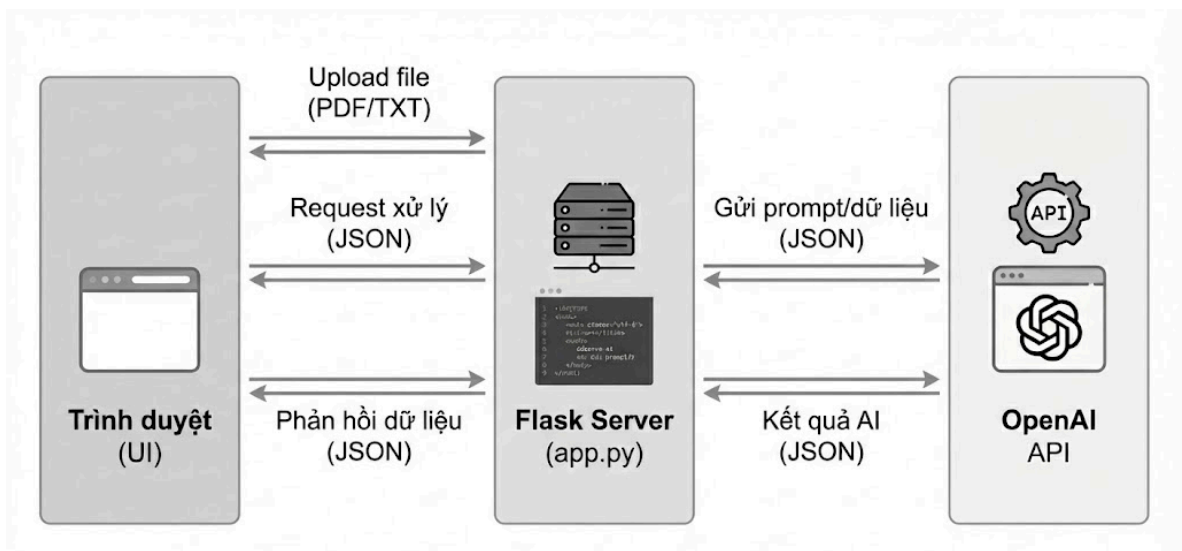
Tầng client (Frontend)

- Thực thi trong trình duyệt.
- Gồm các file: static/index.html, static/styles.css, static/script.js.
- Chịu trách nhiệm:
 - Giao diện người dùng.
 - Quản lý trạng thái (appState).
 - Gửi request tới backend (fetch API).
 - Render tóm tắt, mindmap, flashcard, quiz, JSON import/export.

Tầng server (Backend)

- Chạy bằng Python/Flask (app.py).
- Nhiệm vụ:
 - Serve file tĩnh (index.html, CSS, JS, hình ảnh).
 - Định nghĩa API:
 - /api/process_document
 - /api/generate_quiz

- /api/generate_flashcards
- /api/sessions (GET/POST)
- /api/sessions/<id> (GET)
- Trích xuất văn bản từ file, gọi OpenAI, xử lý JSON và trả kết quả cho client.
- Thành phần lưu trữ: SQLite (file study_sessions.sqlite3) nằm phía backend (không tách thành “tầng 3” độc lập theo triển khai), dùng để lưu session.



Hình 3.7: Sơ đồ kiến trúc khối.

3.3.2. Luồng xử lý chi tiết phía backend

(1) Luồng /api/process_document

1. Nhận request multipart/form-data, field file.
2. Kiểm tra hợp lệ: có file, có filename.
3. Trích xuất text:
 - PDF: đọc bằng PdfReader theo trang,
 - TXT: đọc text.

4. Kiểm tra text rỗng → trả 400 nếu không có nội dung.
5. Gọi `generate_learning_materials(text)`:
 - Cắt `max_chars = 8000`,
 - Tạo messages: `SYSTEM_PROMPT + user content`,
 - Gọi `client.chat.completions.create(model="gpt-4.1-mini")`,
 - Làm sạch output (bỏ code fences), parse JSON,
 - Trả về: `summary, mindmap_mermaid, flashcards, quizzes, source_text`.
6. Trả JSON cho frontend.

(2) Luồng “tạo thêm Flashcard” /api/generate_flashcards

1. Nhận JSON: `context, count, existing_flashcards`.
2. Tách danh sách “front” đã có để chống trùng.
3. Gọi LLM theo `FLASHCARD_ONLY_PROMPT`, yêu cầu trả `{flashcards:[...]}`.
4. Chuẩn hoá difficulty về `easy/medium/hard`, loại bỏ thẻ rỗng.
5. Trả về flashcards mới.

(3) Luồng “tạo thêm Quiz” /api/generate_quiz

1. Nhận JSON: `context, count, existing_quizzes`.
2. Tách danh sách câu hỏi đã có để chống trùng.
3. Gọi LLM theo `QUIZ_ONLY_PROMPT`, yêu cầu trả `{quizzes:[...]}` gồm `mcq/fill/tf`.
4. Parse JSON và trả về quizzes mới.

(4) Luồng DB

- **GET /api/sessions**: select danh sách session, sort mới nhất trước, trả `id/base_name/created_at/source_file/display_name`.

- **POST /api/sessions**: nhận name, source_file, data → serialize JSON → insert DB → trả thông tin session mới.
- **GET /api/sessions/<id>**: load payload_json → parse → trả lại data để frontend render.

3.3.3. Sơ đồ triển khai (Deployment)

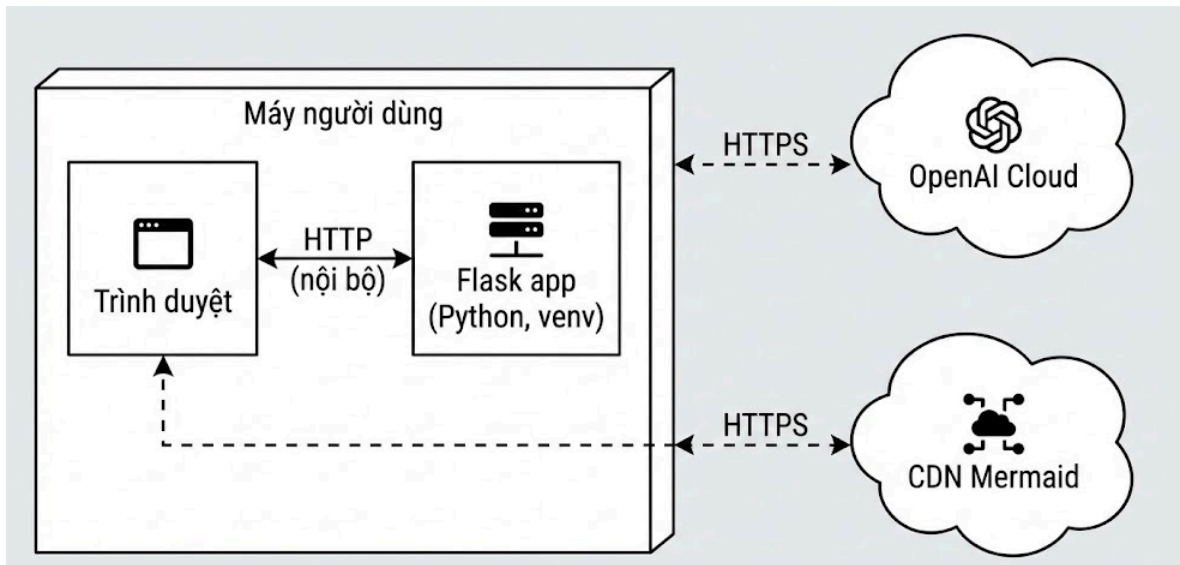
Về triển khai, hệ thống hoạt động trong môi trường đơn giản:

Máy người dùng (local)

- Cài Python 3.x, tạo virtualenv qua setup.bat.
- Chạy ứng dụng bằng python app.py (hoặc run_server.bat).
- Flask chạy ở 0.0.0.0:5000 (chế độ debug).
- Trình duyệt trên cùng máy truy cập <http://localhost:5000>.

Dịch vụ bên ngoài

- Máy local kết nối tới OpenAI API qua Internet.
- Thư viện Mermaid được tải qua CDN (Internet).



Hình 3.8: Sơ đồ deployment.

3.4. Thiết kế dữ liệu & JSON schema

Hệ thống sử dụng 2 dạng dữ liệu chính: (A) JSON học liệu và (B) dữ liệu session trong SQLite.

(A) JSON học liệu (dùng cho hiển thị + import/export + lưu DB)

Payload học liệu thống nhất theo các trường chính:

- summary: string
- mindmap_mermaid: string
- source_text: string (văn bản trích xuất/đã cắt)
- flashcards: mảng {front, back, difficulty}
- quizzes: mảng câu hỏi nhiều loại

Flashcard item

- front: câu hỏi/khái niệm
- back: đáp án/giải thích
- difficulty: easy | medium | hard

Quiz item (3 loại)

- type="mcq":
question, options[], answer (A/B/C/D)
- type="fill":
question (có ____), answer (chuỗi)
- type="tf":
question, answer (boolean true/false)

(B) SQLite – bảng sessions

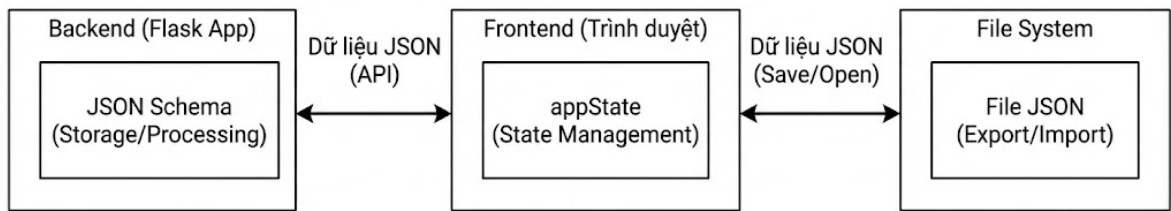
Thiết kế lưu “bản ôn tập” tối giản và phù hợp mục tiêu:

- id (PK, AUTOINCREMENT)
- base_name (tên người dùng nhập)
- created_at (ISO datetime)
- source_file (tên/nguồn file)
- payload_json (chuỗi JSON lưu toàn bộ học liệu)

Ngoài ra tạo index theo created_at để list nhanh theo thời gian.

Display name

- Tạo bởi make_display_name(base_name, created_at) theo định dạng:
<tên> - <dd-mm-YYYY HH:MM>
giúp combobox dễ đọc.



Hình 3.9: Sơ đồ dữ liệu.

3.5. Thiết kế lớp, module và API

3.5.1. Các thành phần backend trong app.py

Backend không tổ chức theo class OOP lớn mà theo **module** + **function** + **route**, phù hợp Flask nhỏ gọn:

(1) Nhóm cấu hình

- load_dotenv() đọc biến môi trường.
- Khởi tạo OpenAI(api_key=..., base_url=...).
- Hằng prompt:
 - SYSTEM_PROMPT: sinh toàn bộ học liệu.
 - FLASHCARD_ONLY_PROMPT: chỉ sinh flashcard.
 - QUIZ_ONLY_PROMPT: chỉ sinh quiz.

(2) Nhóm DB

- get_db_conn(): mở kết nối SQLite.
- init_db(): tạo bảng sessions nếu chưa tồn tại.
- make_display_name(): format tên hiển thị.

(3) Nhóm xử lý văn bản

- extract_text_from_pdf(file) / extract_text_from_file(file): trích xuất text.
- strip_json_code_fences(text): loại bỏ json nếu model lỡ trả về.

(4) Nhóm gọi AI

- `generate_learning_materials(document_text)`: sinh bộ học liệu đầy đủ.
- `generate_more_flashcards(context, existing_fronts, count)`: sinh thêm flashcard.
- `generate_more_quizzes(context, existing_questions, count)`: sinh thêm quiz.

(5) Nhóm API routes

- `/api/process_document`
- `/api/generate_flashcards`
- `/api/generate_quiz`
- `/api/sessions` (GET/POST)
- `/api/sessions/<id>` (GET)

3.5.2. Các thành phần frontend trong script.js

Frontend quản lý toàn bộ UI theo hướng **state-driven**:

(1) Trạng thái trung tâm

- `appState = { summary, mindmap_mermaid, flashcards, quizzes, source_text, source_file }`
- Biến điều khiển flashcard:
 - `currentFlashcardIndex`
 - `showFlashcardAnswer`

(2) Nhóm hàm render

- `renderSummary()`
- `renderMindmap()` (+ hiển thị code)
- `renderFlashcard()` và `renderFlashcardList()` (xem tất cả)
- `renderQuizzes()` (tạo UI theo mcq/fill/tf)

(3) Nhóm xử lý nghiệp vụ

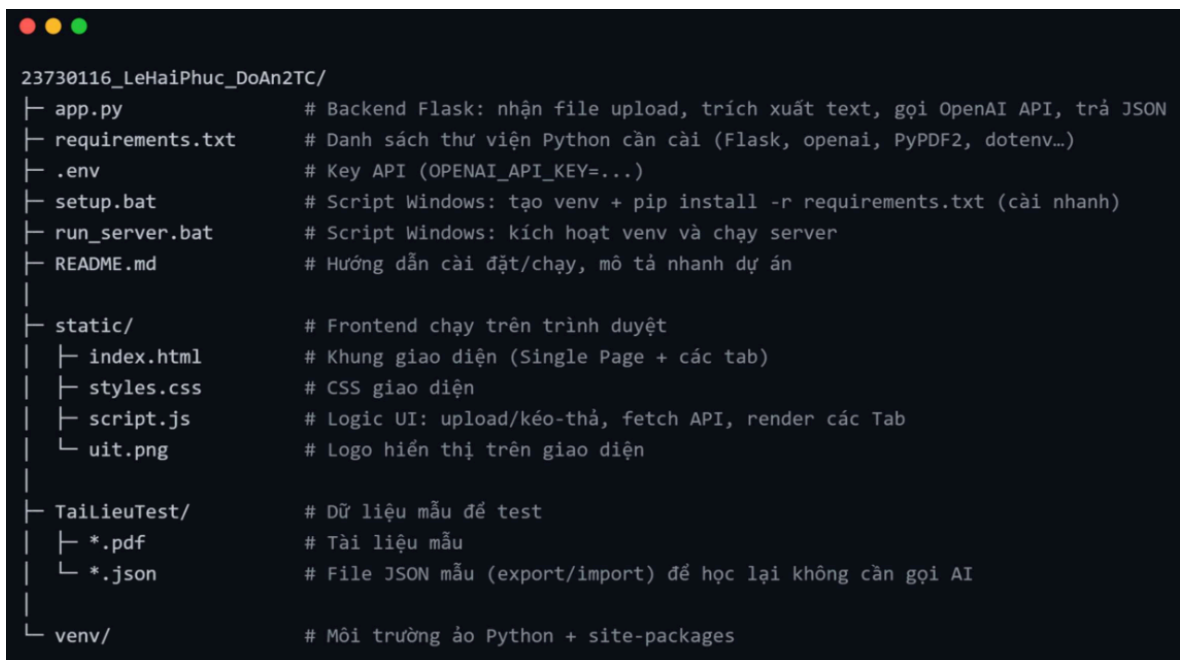
- Upload + xử lý AI:
 - `fetch("/api/process_document", FormData(file))`
- Import/Export JSON:
 - Export tạo payload `{version, exported_at, source_file, source_text, summary, mindmap_mermaid, flashcards, quizzes}`
 - Import đọc JSON và đổ lại vào `appState`
- DB sessions:
 - `setupSavedSessions()` gọi `/api/sessions` để load combobox
 - Change combobox → GET `/api/sessions/<id>`
 - `saveDbBtn` → POST `/api/sessions`

(4) Tạo thêm học liệu

- `buildQuizContext()` / `buildFlashcardContext()`:
 - Ưu tiên `source_text` + `summary`
 - Bổ sung `flashcards/quizzes` (giới hạn số lượng)
 - Chỉ thêm `mindmap` nếu thiếu ngữ cảnh
- `updateAddQuizButtonState()` / `updateAddFlashcardButtonState()` để khóa nút khi context quá ít.

3.6. Cấu trúc thư mục & tổ chức mã nguồn

Cấu trúc chính sau khi giải nén:



Hình 3.10: Cấu trúc thư mục và tổ chức mã nguồn.

3.7. Chuẩn mã hóa, xử lý lỗi & log

Chuẩn mã hóa

- Toàn bộ file frontend dùng charset UTF-8 (<meta charset="UTF-8" />).
- Backend xử lý text:
 - Ưu tiên decode UTF-8.
 - Fallback decode latin-1 (ignore) để tránh crash vì ký tự lạ.

Xử lý lỗi ở backend

- Kiểm tra đầu vào:
 - Thiếu field file → trả HTTP 400.
 - Không trích xuất được text từ file → trả HTTP 500 với thông báo.

- Lỗi JSON từ mô hình:
 - Bắt `json.JSONDecodeError`, trả HTTP 500, thông báo rõ nguyên nhân.
- Lỗi OpenAI:
 - In `repr(e)` ra console.
 - Trả HTTP 500 với thông điệp gợi ý kiểm tra API key/quota.

Xử lý lỗi ở frontend

- Khi không chọn file mà bấm “Xử lý với AI” → hiện alert.
- Khi fetch API trả lỗi → hiển thị thông báo lỗi (alert hoặc status), giữ lại giao diện để người dùng thử lại.
- Khi Mermaid chưa tải được → hiển thị message “Mermaid chưa tải được. Hãy kiểm tra lại kết nối hoặc CDN.”

3.8. Ma trận truy vết yêu cầu

Ma trận dưới đây liên kết Yêu cầu (FR) ↔ Use-case (UC) ↔ Thành phần triển khai:

FR	UC liên quan	Frontend (UI/JS)	Backend (API/Function)	Dữ liệu
FR-01	UC-01	File drop/input, <code>processBtn</code>	<code>/api/process_document</code>	File PDF/TXT
FR-02	UC-01	—	<code>extract_text_from_file()</code>	<code>source_text</code>
FR-03	UC-01	Nhận JSON, set <code>appState</code>	<code>generate_learning_materials()</code>	JSON học liệu
FR-04	UC-02	<code>renderSummary()</code>	—	<code>summary</code>
FR-05	UC-03	<code>renderMindmap()</code> + Mermaid	—	<code>mindmap_mermaid</code>
FR-06	UC-04	<code>renderFlashcard()</code> , prev/next, toggle, show all	—	<code>flashcards[]</code>
FR-07	UC-05	<code>renderQuizzes()</code> , submit + chấm	—	<code>quizzes[]</code>
FR-08	UC-06	<code>exportBtn</code> tạo file JSON	—	export payload
FR-09	UC-07	<code>importFile</code> đọc JSON → set <code>appState</code>	—	import payload
FR-10	UC-08	<code>saveDbBtn</code> (prompt tên)	POST <code>/api/sessions</code>	SQLite <code>sessions</code>
FR-11	UC-09	<code>savedSessionsSelect</code> + refreshList	GET <code>/api/sessions</code>	list sessions
FR-12	UC-09	combobox change → load	GET <code>/api/sessions/<id></code>	payload_json
FR-13	UC-10	<code>addQuizBtn</code> , <code>buildQuizContext()</code>	<code>/api/generate_quiz</code> , <code>generate_more_quizzes()</code>	quizzes mới
FR-14	UC-11	<code>addFlashcardBtn</code> , <code>buildFlashcardContext()</code>	<code>/api/generate_flashcards</code> , <code>generate_more_flashcards()</code>	flashcards mới
FR-15	nhiều UC	update state button/tab	validate + error codes	đảm bảo luồng

Hình 3.11: Ma trận truy vết yêu cầu.

Ma trận này cho thấy mỗi yêu cầu đều có ít nhất một thành phần mã nguồn cụ thể đảm trách, giúp dễ dàng truy vết khi bảo trì hoặc mở rộng.

CHƯƠNG 4. THỰC NGHIỆM

4.1. Môi trường cài đặt và triển khai

4.1.1. Yêu cầu hệ thống

Ứng dụng được triển khai trong môi trường đơn giản, không cần cài thêm CSDL hay server ứng dụng phức tạp. Các yêu cầu tối thiểu như sau:

- **Python:** phiên bản 3.x (khuyến nghị 3.10 trở lên).
- **Hệ điều hành:**
 - Windows (được hỗ trợ tốt với các file .bat đi kèm).
 - Hoặc macOS / Linux (cài đặt thủ công qua terminal).
- **Kết nối Internet ổn định để:**
 - Gọi OpenAI API sinh tóm tắt, mindmap, flashcard, quiz.
 - Tải thư viện Mermaid từ CDN trên frontend để render sơ đồ tư duy.
- **OpenAI API key hợp lệ**, được cấu hình trong file .env ở thư mục gốc.

4.1.2. Chuẩn bị môi trường trên Windows (Cách 1 – dùng script .bat)

Dự án đã chuẩn bị sẵn hai script cho Windows: setup.bat và run_server.bat nhằm tự động hoá các bước tạo môi trường và chạy server.

Quy trình triển khai thực tế:

1. **Giải nén project**
2. **Lần đầu cài đặt – chạy setup.bat**
 - Mở Command Prompt tại thư mục project hoặc double-click setup.bat.
 - Script sẽ lần lượt:
 - Kiểm tra máy đã có Python 3.x chưa.

- Xóa thư mục venv cũ nếu tồn tại.
- Tạo virtualenv mới venv.
- Cài đặt toàn bộ thư viện trong requirements.txt.
- Nhắc người dùng tạo file .env và thiết lập OPENAI_API_KEY.

3. Tạo file .env

Tại thư mục gốc project, tạo file có tên .env với nội dung dạng:

- OPENAI_API_KEY= <openai_api_key>

4. Các lần chạy sau – dùng run_server.bat

- Khi môi trường đã cài xong, chỉ cần:
 - Double-click run_server.bat hoặc chạy trong Command Prompt.
 - Script sẽ kích hoạt venv và gọi python app.py.

5. Truy cập ứng dụng

- Sau khi server báo đã chạy, mở trình duyệt và truy cập:
 - <http://localhost:5000>

```
C:\Windows\System32\cmd.exe
==== 23730116 - Tro ly AI - Setup ====
Xoa virtual environment cu...
Tao virtual environment moi...

Nang cap pip va cai thu vien...
Requirement already satisfied: pip in c:\users\finn\downloads\23730116_lehaiphuc_doan2tc - copy\venv\lib\site-packages (25.2)
Collecting pip
  Using cached pip-25.3-py3-none-any.whl.metadata (4.7 kB)
Using cached pip-25.3-py3-none-any.whl (1.8 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.2
    Uninstalling pip-25.2:
      Successfully uninstalled pip-25.2
Successfully installed pip-25.3
Collecting flask (from -r requirements.txt (line 1))
  Using cached flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting openai>=1.0.0 (from -r requirements.txt (line 2))
  Using cached openai-2.8.1-py3-none-any.whl.metadata (29 kB)
Collecting python-dotenv (from -r requirements.txt (line 3))
  Using cached python_dotenv-1.2.1-py3-none-any.whl.metadata (25 kB)
Collecting PyPDF2 (from -r requirements.txt (line 4))
  Using cached pypdf2-3.0.1-py3-none-any.whl.metadata (6.8 kB)
Collecting blinker>=1.9.0 (from flask->-r requirements.txt (line 1))
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask->-r requirements.txt (line 1))
  Using cached click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask->-r requirements.txt (line 1))
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)

C:\Windows\System32\cmd.exe
Using cached markupsafe-3.0.3-cp314-cp314-win_amd64.whl (15 kB)
Using cached sniffio-1.3.1-py3-none-any.whl (10 kB)
Using cached tqdm-4.67.1-py3-none-any.whl (78 kB)
Using cached typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Using cached werkzeug-3.1.3-py3-none-any.whl (224 kB)
Using cached certifi-2025.11.12-py3-none-any.whl (159 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: typing-extensions, sniffio, python-dotenv, PyPDF2, markupsafe, jiter, itsdangerous, idna, h11, distro, colorama, certifi, blinker, annotated-types, werkzeug, typing-inspection, tqdm, pydantic-core, Jinja2, httpcore, click, anyio, pydantic, httpx, flask, openai
Successfully installed PyPDF2-3.0.1 annotated-types-0.7.0 anyio-4.11.0 blinker-1.9.0 certifi-2025.11.12 click-8.3.1 colorama-0.4.6 distro-1.9.0 flask-3.1.2 h11-0.16.0 httpcore-1.0.9 httpx-0.28.1 idna-3.11 itsdangerous-2.2.0 Jinja2-3.1.6 jiter-0.12.0 markupsafe-3.0.3 openai-2.8.1 pydantic-2.12.4 pydantic-core-2.41.5 python-dotenv-1.2.1 sniffio-1.3.1 tqdm-4.67.1 typing-extensions-4.15.0 typing-inspection-0.4.2 werkzeug-3.1.3

Who tao file .env va dat OPENAI_API_KEY=...

* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.153:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 887-894-109
```

Hình 4.1: Kết quả chạy setup.bat.

4.1.3. Chuẩn bị môi trường thủ công (Windows / macOS / Linux)

Ngoài cách dùng script, đây là cách cài đặt thủ công, phù hợp trên mọi hệ điều hành:

1. Di chuyển vào thư mục project: `cd 23730116_LeHaiPhuc_DoAn2TC`

2. Tạo virtualenv: `python -m venv venv`

3.Kích hoạt virtualenv

- Windows: `venv\Scripts\activate`
- macOS / Linux: `source venv/bin/activate`

4.Cài đặt thư viện

```
pip install --upgrade pip
```

```
pip install -r requirements.txt
```

5.Tạo file .env cấu hình API key:

```
OPENAI_API_KEY=your_openai_api_key_here
```

6.Chạy server Flask: `python app.py`

7.Mở trình duyệt: Truy cập `http://localhost:5000` để sử dụng ứng dụng.

4.1.4. Kiểm tra khi khởi động & xử lý sự cố thường gặp

1. Không chọn tệp nhưng bấm “Xử lý với AI”

- Frontend kiểm tra biến `selectedFile`, nếu rỗng sẽ hiện thông báo:
 - Vui lòng chọn tệp trước.
- Không gửi request lên server, giúp tránh lỗi không cần thiết.

2. Thiếu trường file ở request

- Nếu frontend gửi request thiếu field `file` (hoặc request không đúng cấu trúc), backend trả về:
 - HTTP 400 với thông báo:

- "Không tìm thấy tệp upload (field name 'file')."

3. Tên file trống hoặc lỗi đọc file

- Backend kiểm tra file.filename. Nếu trống → trả HTTP 400:
 - "Tên tệp trống."
- Nếu lỗi khi đọc nội dung (ví dụ file hỏng) → HTTP 500 với thông báo:
 - "Lỗi khi đọc tệp: ..."

4. Không trích xuất được nội dung văn bản

- Hàm `extract_text_from_file()` xử lý PDF bằng PdfReader hoặc decode text.
- Nếu kết quả trống → trả về lỗi:
 - "Không trích xuất được nội dung văn bản từ tệp."

5. Mô hình trả về JSON không hợp lệ

- Khi `json.loads(content)` bị `JSONDecodeError`, backend trả HTTP 500:
 - "Mô hình trả về JSON không hợp lệ. Hãy thử lại."
- Frontend bắt lỗi, hiển thị status màu đỏ và nội dung thông báo lỗi ở vùng trạng thái upload.

6. Lỗi OpenAI API (API key, quota, mạng)

- Bắt kỳ lỗi nào khi gọi `client.chat.completions.create(...)` được bắt và log ra console (server).
- Client nhận thông báo chung:

- "Lỗi khi gọi OpenAI API. Kiểm tra API key / quota."

7. Mermaid không tải được (lỗi CDN / mạng)

- Trong renderMindmap(), nếu window.mermaid không khả dụng, frontend hiển thị đoạn text:
 - "Mermaid chưa tải được. Hãy kiểm tra lại kết nối hoặc CDN."
- Giúp người dùng hiểu đây là lỗi về thư viện phía client, không phải lỗi backend.

4.2. Thiết lập dữ liệu mẫu và kịch bản kiểm thử

4.2.1. Đặc điểm dữ liệu mẫu dùng trong thử nghiệm

Ứng dụng được thiết kế để làm việc tốt nhất với tài liệu học tập dạng văn bản ở hai định dạng chính:

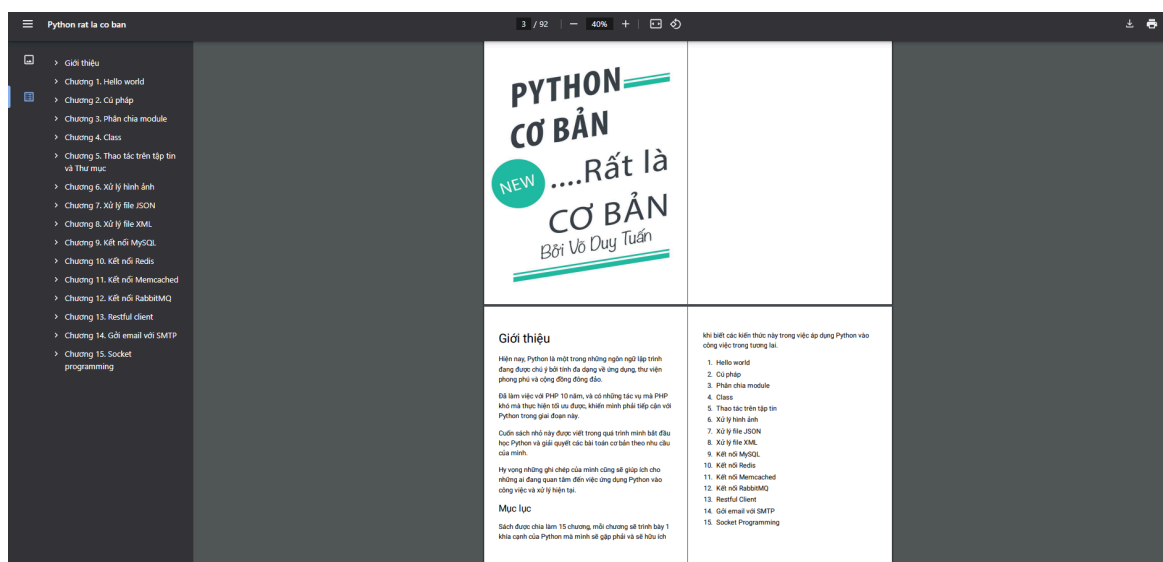
- **PDF**: giáo trình, slide xuất ra PDF, tài liệu môn học.
- **TXT / text thuần**: ghi chú, trích xuất sẵn từ tài liệu khác.

Trong quá trình sử dụng và kiểm thử, có thể lựa chọn các tài liệu mẫu mang tính điển hình cho bối cảnh đề tài, chẳng hạn:

- Một chương giáo trình hoặc bài giảng (PDF) để kiểm tra khả năng tóm tắt và sinh quiz.
- Một file TXT chứa ghi chú, tóm lược nội dung môn học để kiểm tra flashcard và mindmap.

Điểm chung khi chuẩn bị dữ liệu mẫu:

- Nội dung ưu tiên tiếng Việt, phù hợp với prompt và yêu cầu “mọi nội dung là tiếng Việt nếu tài liệu là tiếng Việt” trong SYSTEM_PROMPT.
- Tài liệu có cấu trúc rõ (tiêu đề, mục, gạch đầu dòng) sẽ giúp mô hình dễ trích rút ý chính và tổ chức mindmap tốt hơn.



Hình 4.2: PDF mẫu thử nghiệm (truy cập tại

<https://github.com/TienNHM/ebooks/blob/master/Python/Python%20rat%20la%20co%20ba%20-%20Vo%20Duy%20Tuan.pdf>).

4.2.2. Kịch bản kiểm thử

Kịch bản 1 – Upload và xử lý tài liệu thành công

- **Mục tiêu:** kiểm tra toàn bộ pipeline: upload → xử lý với AI → hiển thị 4 dạng nội dung.
- **Bước thực hiện:**
 1. Mở trình duyệt, truy cập <http://localhost:5000>.
 2. Tại tab Trang chủ, chọn một file PDF/TXT hợp lệ.

3. Kiểm tra trạng thái upload hiển thị tên file và thông báo đã tải thành công.
4. Nhấn “Xử lý với AI”.
5. Quan sát dòng trạng thái chuyển sang “Đang xử lý tài liệu với AI...” và nút hiển thị spinner.
6. Đợi đến khi status chuyển sang “Xử lý xong! Bạn có thể mở các tab để xem kết quả.”, đồng thời các tab Tóm tắt, Mindmap, Flashcard, Quiz được kích hoạt.
7. Mở lần lượt 4 tab và xác nhận:
 - Tab Tóm tắt có nội dung không rỗng, bám sát tài liệu.
 - Tab Mindmap hiển thị sơ đồ (SVG) và có code Mermaid trong khung code.
 - Tab Flashcard có danh sách thẻ, duyệt được Prev/Next, ẩn/hiện đáp án.
 - Tab Quiz hiển thị danh sách câu hỏi và có thể nộp bài.
8. Tạo thêm quiz và flashcard.

Kịch bản 2 – Không chọn tệp nhưng bấm xử lý

- **Mục tiêu:** đảm bảo hệ thống không gửi request sai và có thông báo rõ ràng.
- **Bước thực hiện:**
 1. Mở ứng dụng, không chọn file.
 2. Bấm nút “Xử lý với AI”.

3. Xác nhận xuất hiện hộp thoại alert với nội dung:

■ "Vui lòng chọn tệp trước."

4. Không có request nào được gửi lên `/api/process_document` (có thể kiểm tra bằng tab Network của trình duyệt).

Kịch bản 3 – Lỗi khi đọc file / file không hợp lệ

- **Mục tiêu:** quan sát cách hệ thống phản hồi khi backend không trích xuất được text.
- **Gợi ý thực hiện:**
 - Dùng một file không phải PDF/text (hoặc file PDF hỏng) để thử upload.
 - Quan sát:
 - Backend trả về thông báo lỗi tương ứng ("Lỗi khi đọc tệp: ...", "Không trích xuất được nội dung văn bản từ tệp").
 - Frontend hiển thị status màu đỏ với message lỗi.

Kịch bản 4 – Lỗi OpenAI (API key sai hoặc hết hạn)

- **Mục tiêu:** đảm bảo thông báo lỗi phía người dùng rõ ràng khi không gọi được mô hình.
- **Bước thực hiện (thử nghiệm có chủ đích):**
 1. Thiết lập một API key không hợp lệ trong `.env`.
 2. Chạy ứng dụng, upload file hợp lệ, bấm "Xử lý với AI".
 3. Xác nhận backend log lỗi và trả về thông báo:

- "Lỗi khi gọi OpenAI API. Kiểm tra API key / quota."

4. Frontend hiển thị thông báo lỗi này trong vùng trạng thái.

Kịch bản 5 – Kiểm tra chức năng Lưu vào CSDL và Nhập/Xuất JSON

- **Mục tiêu:** đảm bảo người dùng có thể lưu và tải lại kết quả xử lý mà không cần gọi AI lần nữa và chia sẻ tài liệu.
- **Bước thực hiện:**
 1. Thực hiện Kịch bản 1 để có một bộ kết quả đầy đủ.
 2. Cuộn xuống khu vực Lưu vào CSDL và Nhập/Xuất dữ liệu, bấm “Xuất dữ liệu JSON” hoặc Lưu vào CSDL.
 3. Kiểm tra file .json được tải xuống chứa đủ các trường summary, mindmap_mermaid, flashcards, quizzes, source_file.
 4. Tải lại trang, *không* xử lý AI.
 5. Dùng chức năng “Nhập dữ liệu từ JSON” để chọn file vừa tải hoặc chọn dữ liệu đã lưu trong combobox CSDL.
 6. Kiểm tra các tab kết quả hiển thị lại đúng nội dung như trước khi export.

Dưới đây là bảng kịch bản kiểm thử:

TC	Mục tiêu	Dữ liệu	Bước thực hiện	Kết quả mong đợi
----	----------	---------	----------------	------------------

TC-01	Upload + xử lý PDF	Tài liệu PDF	Home → chọn PDF → “Xử lý với AI”	Tab Summary/Mindmap/Flashcard/Quiz được enable; có dữ liệu hiển thị
TC-02	Upload + xử lý TXT	file .txt bất kỳ	Chọn TXT → xử lý	Kết quả sinh ra tương tự PDF
TC-03	Hiển thị Mindmap	PDF đã xử lý	Vào tab Mindmap	Mermaid render được; copy code hoạt động
TC-04	Ôn flashcard	PDF đã xử lý	Tab Flashcard → Next/Prev → Hiện/Ẩn đáp án → Xem tất cả	Điều hướng đúng; hiển thị badge độ khó
TC-05	Làm quiz MCQ + Fill	PDF đã xử lý	Tab Quiz → chọn đáp án/nhập → Nộp bài	Chấm điểm; hiển thị đúng/sai, tổng điểm %
TC-06	Quiz Đúng/Sai (TF)	PDF đã xử lý	Tab Quiz → gấp câu TF → chọn Đúng/Sai → Nộp bài	Chấm đúng theo answer=true/false
TC-07	Xuất JSON	Có học liệu	Nhấn “Xuất ra file JSON”	Tải được file *_study_data.json đúng schema
TC-08	Nhập JSON	File JSON mẫu	Nhấn “Nhập từ file JSON”	Tự đổ dữ liệu vào các tab; ôn tập được ngay

TC-09	Lưu vào CSDL	Có học liệu	Nhấn “Lưu vào CSDL” → nhập tên	Tạo session; combobox có item mới (kèm thời gian)
TC-10	Tải từ CSDL	Có session đã lưu	Chọn item trong combobox	Dữ liệu nạp lại đầy đủ; UI cập nhật trạng thái “Đã tải bản ôn tập...”
TC-11	Tạo thêm Quiz	Có summary/flashcards	Tab Quiz → “+ Tạo thêm Quiz” → nhập số lượng	Sinh thêm quiz; danh sách quiz tăng lên
TC-12	Tạo thêm Flashcard	Có summary/quiz	Tab Flashcard → “+ Tạo thêm Flashcard” → nhập số lượng	Sinh thêm flashcard; danh sách flashcard tăng lên
TC-13	Kiểm thử lỗi thiếu context	Chưa xử lý gì	Bấm “Tạo thêm Quiz/Flashcard”	Nút bị disable hoặc báo “Chưa đủ dữ liệu...”
TC-14	Kiểm thử lỗi API key	Xóa API key	Chạy app	App báo thiếu OPENAI_API_KEY, không chạy

Bảng 4.1: Kịch bản kiểm thử chấp thuận.

4.3. Kết quả thực nghiệm

4.3.1. Kết quả chức năng

Từ các kịch bản kiểm tra, có thể tổng hợp mức độ hoàn thành chức năng như sau:

- **Pipeline chính “Upload → Xử lý với AI → Hiển thị kết quả” hoạt động trơn tru:**

- Người dùng chọn tệp PDF/TXT, trạng thái upload được cập nhật rõ ràng.
- Khi xử lý, giao diện hiển thị “Đang xử lý tài liệu với AI...” kèm spinner, tránh cảm giác “đơ” ứng dụng.
- Sau khi nhận JSON từ backend, bốn trường summary, mindmap_mermaid, flashcards, quizzes được lưu vào appState và sử dụng để render các tab tương ứng.

- **Các tab nội dung thể hiện đúng vai trò:**

- Tab Tóm tắt: hiển thị bản rút gọn nội dung tài liệu, dễ đọc, dễ lướt.
- Tab Mindmap: render code Mermaid thành sơ đồ tư duy, đồng thời cho phép copy code phục vụ các công cụ khác.
- Tab Flashcard:
 - Duyệt từng thẻ bằng Prev/Next.
 - Ẩn/hiện đáp án linh hoạt.
 - Thể hiện độ khó qua badge màu.
 - Grid “Xem tất cả thẻ” giúp ôn tập nhanh.
- Tab Quiz:
 - Kết hợp câu trắc nghiệm, đúng/sai và điền khuyết.
 - Sau khi nộp bài, hiển thị số câu đúng và feedback chi tiết.

- **Chức năng Lưu vào CSDL và Nhập/Xuất JSON hoạt động ổn định:**

- Giúp tránh gọi lại AI cho cùng một tài liệu, tiết kiệm chi phí và thời gian.
- Đảm bảo khả năng chia sẻ bộ tóm tắt/flashcard/quiz sang máy khác miễn có file JSON.

Ngoài ra còn có nhóm chức năng mở rộng học liệu

- “Tạo thêm Quiz”: gửi context tổng hợp (source_text + summary + tối đa 30 flashcard) và danh sách câu hỏi đã có để giảm trùng.
- “Tạo thêm Flashcard”: gửi context (source_text + summary + tối đa 20 câu quiz) và danh sách front đã có để giảm trùng.

Nhìn chung, các chức năng chính được hiện thực đúng với mục tiêu đề tài: từ một tài liệu đầu vào, hệ thống cung cấp đầy đủ bốn “gói” nội dung phục vụ học tập và cho phép tương tác trực tiếp trên web.

4.3.2. Quan sát về hiệu năng

Trong phạm vi đồ án, quá trình đo kiểm hiệu năng được hiểu theo nghĩa thời gian phản hồi cảm nhận được đối với người dùng cuối, thay vì các chỉ số phức tạp (TPS, throughput, benchmark tải lớn). Một số điểm rút ra từ thiết kế và quan sát thực tế:

- **Độ trễ chủ yếu đến từ OpenAI API**

- Backend Flask chỉ đảm nhiệm các thao tác nhẹ: nhận file, trích xuất text, gửi/nhận JSON.
- Thời gian xử lý phụ thuộc nhiều vào:
 - Kích thước tài liệu (văn bản sau khi trích xuất).

- Hiệu suất của mô hình gpt-4.1-mini tại thời điểm gọi.
- Tốc độ kết nối Internet.
- **Giới hạn độ dài văn bản (khoảng 8.000 ký tự)**
 - Giúp tránh vượt giới hạn ngữ cảnh của mô hình.
 - Giảm nguy cơ thời gian phản hồi quá dài hoặc request thất bại.
- **Hiệu năng đủ cho tương tác cá nhân**
 - Ứng dụng hướng tới quy mô một người dùng / một tài liệu tại một thời điểm (mô hình “trợ lý học tập cá nhân”), nên không đặt mục tiêu chịu tải đồng thời cao.
 - Với cách triển khai hiện tại (Flask chạy chế độ dev, không load balancing), hiệu năng là phù hợp với phạm vi đồ án và nhu cầu thử nghiệm.

Đo thực nghiệm

1. Mở Chrome DevTools → tab **Network**.
2. Thực hiện xử lý tài liệu → click vào request `/api/process_document`.
3. Ghi nhận:
 - Waiting (TTFB) và Content Download,
 - Tổng thời gian request.
4. Thực hiện tương tự với `/api/generate_quiz` và `/api/generate_flashcards`.

Bảng ghi nhận thời gian

Thao tác	Dữ liệu	Kích thước file	Thời gian trung bình (s)	Ghi chú
Xử lý tài liệu	PDF 1	~1.0MB	45s	phụ thuộc mạng/API
Xử lý tài liệu	PDF 2	~1.3MB	30s	phụ thuộc độ dài text
Tạo thêm Quiz (6 câu)	context từ summary+flashcards	—	20s	request /api/generate_quiz
Tạo thêm Flashcard (8 thẻ)	context từ summary+quiz	—	20s	request /api/generate_flashcards

Bảng 4.2: Bảng đo thời gian.

Nhận xét thực nghiệm

- Độ trễ chủ yếu nằm ở bước gọi API (phần sinh nội dung).
- Các thao tác thuần frontend (render tab, lật flashcard, chấm quiz) phản hồi tức thời.

4.3.3. Kiểm chứng bảo mật & nhất quán dữ liệu

Về mặt bảo mật và nhất quán, đồ án không triển khai các cơ chế phức tạp (auth, phân quyền, mã hoá dữ liệu lưu trữ), nhưng vẫn đảm bảo được một số nguyên tắc cơ bản phù hợp phạm vi:

- **Không lưu nội dung tài liệu và kết quả trên server**
 - Backend xử lý file và nội dung text trong bộ nhớ của request hiện tại.
 - Sau khi trả JSON cho client, dữ liệu không được lưu vào CSDL hay file tạm trên server.
- **API key được tách khỏi mã nguồn**
 - Sử dụng .env để lưu OPENAI_API_KEY.
 - Mã nguồn không hard-code API key, tránh lộ khoá khi chia sẻ project.
- **Thông báo lỗi rõ ràng nhưng không tiết lộ chi tiết nhạy cảm**
 - Các thông báo tới client (ví dụ: “Lỗi khi gọi OpenAI API. Kiểm tra API key / quota.”) mang tính khái quát, không in ra stack trace hay thông tin nội bộ.
 - Thông tin chi tiết hơn (như repr(e)) chỉ log ở console server – nơi nhà phát triển kiểm tra.
- **Nhất quán dữ liệu giữa backend – frontend**
 - JSON schema được cố định (summary, mindmap_mermaid, flashcards, quizzes).
 - Frontend luôn kiểm tra kiểu dữ liệu (ví dụ: `Array.isArray(data.flashcards) ? ... : []`) trước khi render, tránh lỗi khi mô hình trả về thiếu trường.

Những cơ chế này chưa đủ cho một hệ thống nhiều người dùng, nhưng phù hợp với bản demo đồ án, đồng thời đặt nền tảng tốt để bổ sung các lớp bảo mật cao hơn (đăng nhập, HTTPS, token...) trong các phiên bản tiếp theo.

4.4. So sánh các kỹ thuật và đánh giá độ chính xác.

4.4.1. Bảng so sánh: Gọi API AI vs kỹ thuật Text Ranking (không dùng AI)

Ngay từ đầu, dự án dự kiến triển khai theo hướng Text Ranking (Extractive NLP), trong đó hệ thống xếp hạng câu và từ khoá theo mức độ quan trọng/độ liên quan dựa trên NLP truyền thống (tách câu, tokenization/word segmentation) kết hợp thuật toán thống kê như TF-IDF và MMR (Maximal Marginal Relevance) để trích xuất nội dung tóm tắt và cấu trúc ôn tập.

Tuy nhiên, sau giai đoạn thử nghiệm và so sánh với phương án LLM thông qua API, đồ án đã chuyển sang hướng sử dụng LLM API nhằm cải thiện chất lượng diễn đạt và khả năng tổng hợp. Dưới đây là bảng so sánh chi tiết:

Tiêu chí	Gọi API AI (LLM)	Text ranking không dùng AI <i>(Extractive NLP) dựa trên thống kê và luật (rule-based)</i>
Chất lượng tóm tắt	Thường mạch lạc, gom ý, diễn đạt tự nhiên	Thường là “cắt-dán” câu top-k, dễ rời rạc
Flashcard/Quiz	Tạo được câu hỏi đa dạng (MCQ/TF/Fill), diễn đạt linh hoạt	Khó tạo câu hỏi tự nhiên; thường phụ thuộc template → đơn điệu
Tính bám nguồn	Dễ bị suy diễn nhưng có thể khắc phục bằng prompt ràng buộc, chặt chẽ.	Bám nguồn cao vì chủ yếu trích xuất từ câu gốc
Khả năng bao phủ nội dung	Tốt khi context đủ; nhưng bị giới hạn bởi max_chars nhưng dễ khắc phục bằng kỹ thuật chunking.	Phụ thuộc vào thuật toán ranking; dễ bỏ sót kiến thức ngầm
Yêu cầu triển khai	Dễ (gọi API) nhưng phụ thuộc Internet và key	Tự chạy offline, không cần key
Chi phí vận hành	Có chi phí thấp theo token	Gần như 0 (chỉ CPU)
Hiệu năng	Phụ thuộc mạng và API	Nhanh, offline
Khả năng mở rộng	Mở rộng dạng học liệu dễ (chỉ đổi prompt/schema)	Mở rộng khó (phải viết thuật toán/luật mới)

Phù hợp mục tiêu “tạo học liệu phong phú”	Rất phù hợp	Hạn chế
---	-------------	---------

Bảng 4.3: Bảng so sánh giữa hai kỹ thuật.

Dưới đây là một số hình ảnh so sánh đầu ra giữa hai kỹ thuật nói trên:



Text ranking VS LLM AI

Hình 4.3: So sánh đầu ra giữa Text ranking và AI.

Kết luận so sánh

- Nếu mục tiêu là học liệu phong phú, dễ học, đa dạng câu hỏi → LLM phù hợp hơn.
- Nếu mục tiêu là offline, chi phí 0, bám nguồn tuyệt đối → text ranking phù hợp nhưng chất lượng học liệu (đặc biệt quiz/flashcard) thường kém tự nhiên.

4.4.2. Đánh giá độ chính xác.

Đánh giá độ chính xác theo 3 trục: Correctness – Groundedness – Usefulness. Để đánh giá chất lượng học liệu sinh ra một cách thuyết phục, thay vì chỉ hỏi “đúng hay sai”, đề tài áp dụng thang điểm 0–2 theo 3 tiêu chí:

(1) Correctness (Đúng kiến thức)

- 0: sai
- 1: đúng một phần / mơ hồ
- 2: đúng rõ ràng

(2) Groundedness (Bám nguồn)

- 0: không tìm thấy trong tài liệu
- 1: có liên quan nhưng suy diễn nhiều
- 2: chỉ ra được đoạn trong tài liệu chứng minh

(3) Usefulness (Hữu ích để ôn tập)

- 0: vô dụng / quá dễ / quá mơ hồ
- 1: dùng được nhưng chưa tối ưu
- 2: rất tốt, đúng trọng tâm

Chuẩn bị dữ liệu PDF:



Hình 4.2: PDF mẫu thử nghiệm (truy cập tại

<https://github.com/TienNHM/ebooks/blob/master/Python/Python%20rat%20la%20co%20ba%20-%20Vo%20Duy%20Tuan.pdf>).

Bảng 4.4: đánh giá Summary:

ID	Ý tóm tắt được chọn	Trích dẫn nguồn	C	G	U	Total (0–6)
S-01	Python là ngôn ngữ phiên dịch, chạy trực tiếp không cần build file thực thi (so sánh như PHP).	Trang 6, đoạn mở đầu “Python là một ngôn ngữ phiên dịch...”	2	2	2	6
S-02	Python có 2 nhánh chính: 2.x (dừng phát triển, 2.7) và 3.x (tiếp tục phát triển).	Trang 6, đoạn “Hiện tại Python có 2 nhánh...”	2	2	2	6
S-03	Nhiều cấu trúc điều khiển dựa vào thụt đầu dòng (indentation) để tạo block thay vì {...}.	Trang 11, đoạn “dựa vào thụt đầu dòng (indentation)...”	2	2	2	6
S-04	Python không có cấu trúc switch...case.	Trang 12, mục “2.4.2. Switch...case”	2	2	1	5

S-05	Hàm nếu không return thì mặc định trả về None.	Trang 15, đoạn “Hàm nếu không trả dữ liệu...”	2	2	2	6
------	--	---	---	---	---	---

Kết quả:

- $\text{Correctness_avg} = (2+2+2+2+2)/5 = 10/5 = \mathbf{2.0}$
- $\text{Groundedness_avg} = (2+2+2+2+2)/5 = 10/5 = \mathbf{2.0}$
- $\text{Usefulness_avg} = (2+2+2+1+2)/5 = 9/5 = \mathbf{1.8}$
- $\text{Total_avg} = (6+6+6+5+6)/5 = 29/5 = \mathbf{5.8}$

Nhận xét: Summary bám sát kiến thức và trích được nguồn rõ ràng. Một ý về “switch...case” có U=1 vì là thông tin đơn lẻ, ít “ra ý chính” nếu đứng riêng.

Bảng 4.5: đánh giá Flashcards:

ID	Chủ đề thẻ	Trích dẫn nguồn	C	G	U	Total
FC-01	Python là ngôn ngữ Interpreter là gì?	Tr.6, đoạn “ngôn ngữ biên dịch...”	2	2	2	6
FC-02	Lệnh kiểm tra phiên bản Python (python --version).	Tr.6, phần “Cài đặt... gõ: \$ python --version”	2	2	2	6
FC-03	Hai nhánh Python 2.x và 3.x khác nhau thế nào (trạng thái phát triển).	Tr.6, đoạn “2.x... 2.7; 3.x... tiếp tục”	2	2	2	6
FC-04	Boolean và toán tử logic: True/False, not/and/or.	Tr.10, mục “2.3. Boolean và Toán tử logic”	2	2	2	6
FC-05	So sánh kép kiểu $1 < x < 3$ nghĩa là gì?	Tr.11, các ví dụ so sánh kép	2	1	2	5
FC-06	Hai phong cách kiểm tra index List: LBYL vs EAFP.	Tr.24, mục “Look before you leap (LBYL)” và “EAFP”	2	2	2	6

FC-07	Cắt chuỗi (slicing) ví dụ str[0:4], str[-3:].	Tr.19, mục “Trích xuất chuỗi con”	2	2	2	6
FC-08	list.append() và list.pop() dùng để làm gì?	Tr.27–28, mục “append” và “pop”	2	2	2	6
FC-09	open(filepath, mode) và các mode r/w/a/r+/w+/a+, thêm b cho binary.	Tr.45–46, mục “Mở file”	2	2	2	6
FC-10	Pipeline trong Redis để tăng performance (gộp nhiều lệnh).	Tr.73–74, mục “10.4. Pipeline”	2	1	1	4

Kết quả:

- Correctness_avg = (tổng C = 20)/10 = **2.0**
- Groundedness_avg = (tổng G = 18)/10 = **1.8**
- Usefulness_avg = (tổng U = 19)/10 = **1.9**
- Total_avg = (tổng Total = 57)/10 = **5.7**

Nhận xét : Flashcards nhìn chung rất tốt (Total_avg 5.7). Một vài thẻ có G=1 vì nội dung thường tóm ý từ nhiều dòng/ví dụ liên tiếp, khó trả đúng 1 câu đơn lẻ.

Bảng 4.6: đánh giá Quizzes:

ID	loại	Mô tả câu hỏi	Trích dẫn nguồn	C	G	U	tổng
Q-01	mcq	Lệnh nào kiểm tra phiên bản Python?	Tr.6, phần “\$ python --version”	2	2	2	6
Q-02	tf	“Python cần build thành file thực thi trước khi chạy.” (Đ/S)	Tr.6, đoạn “không cần build... chạy trực tiếp”	2	2	2	6
Q-03	fill	Điền: Giá trị đúng/sai trong Python là ____ và ____.	Tr.10, mục “True và False”	2	2	2	6
Q-04	mcq	Python dùng gì để tạo block xử lý trong cấu trúc điều khiển?	Tr.11, đoạn “thụt đầu dòng...”	2	2	2	6

Q-05	tf	“Python có cấu trúc switch...case.” (Đ/S)	Tr.12, mục “Python không có...”	2	2	1	5
Q-06	fill	Điền: Hàm không return thì mặc định trả về ____.	Tr.15, đoạn “mặc định... None”	2	2	2	6
Q-07	mcq	Với str='Hello world', cách lấy 3 ký tự cuối là gì?	Tr.19, ví dụ print str[-3:]	2	1	2	5
Q-08	fill	Điền: Hàm loại bỏ khoảng trắng 2 đầu chuỗi là ____.	Tr.22, mục strip([chars])	2	2	1	5
Q-09	tf	“list.reverse() trả về list mới, không đổi list cũ.” (Đ/S)	Tr.29–30, mục “reverse()... không trả về... thay đổi trực tiếp”	2	2	2	6
Q-10	mcq	Mode nào mở file để đọc + ghi thêm cuối file (con trỏ ở cuối)?	Tr.46, mục “a+ : mở để đọc và thêm...”	2	2	1	5

Kết quả:

- Correctness_avg = (tổng C = 20)/10 = **2.0**
- Groundedness_avg = (tổng G = 19)/10 = **1.9**
- Usefulness_avg = (tổng U = 17)/10 = **1.7**
- Total_avg = (tổng Total = 56)/10 = **5.6**

Nhận xét: Quiz bám nội dung tốt; một vài câu “dễ” nên U=1 (dùng để kiểm tra kiến thức cơ bản/nhớ nhanh).

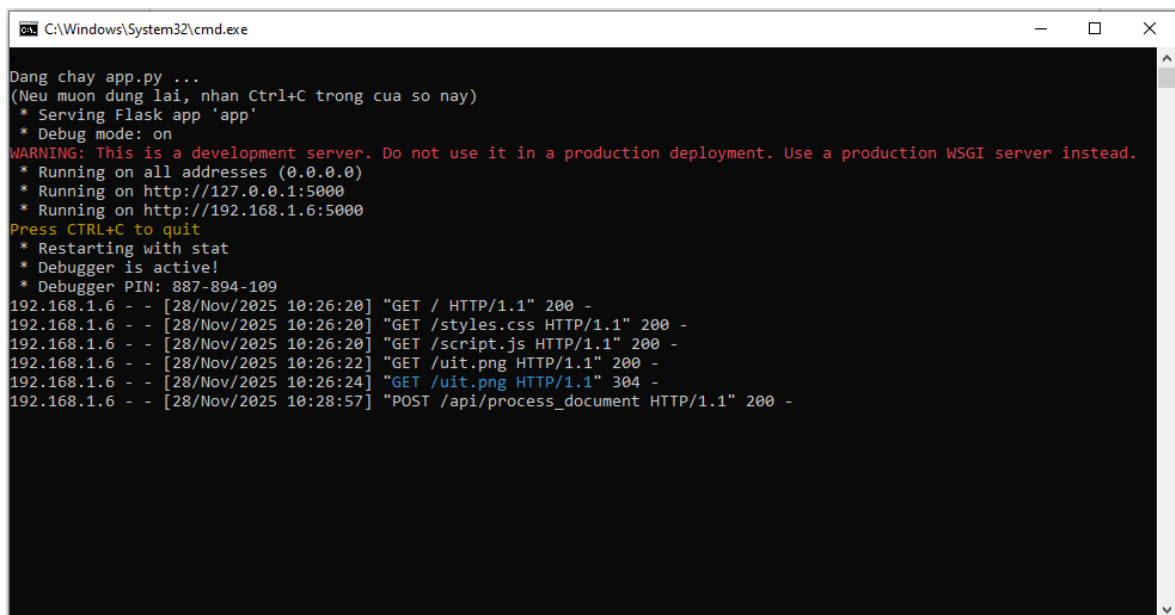
Kết luận: Từ đó có thể thấy rằng đề án sử dụng LLM API nhưng kết quả đầu ra khá chính xác, không bị suy diễn và ảo tưởng nhờ kỹ thuật prompt chặt chẽ, ràng buộc kết quả đầu ra.

4.5. Hướng dẫn sử dụng nhanh, những hình ảnh web thực tế.

Phần này tổng hợp lại luồng thao tác cơ bản thành một bản tóm tắt nhanh để người dùng mới có thể bắt đầu nhanh chóng:

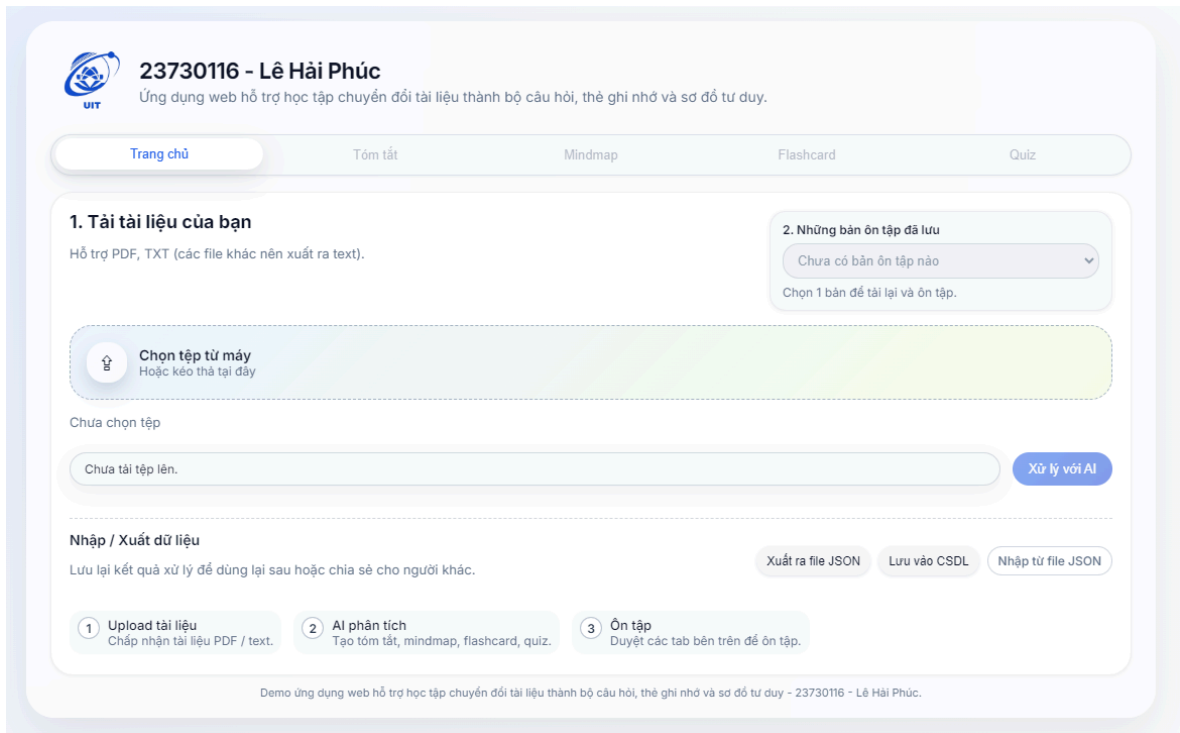
1. Khởi động ứng dụng

- Chạy setup.bat (lần đầu) hoặc run_server.bat (các lần sau) trên Windows.
- Hoặc kích hoạt venv và chạy python app.py trên terminal.
- Mở trình duyệt truy cập: <http://localhost:5000>.



```
C:\Windows\System32\cmd.exe

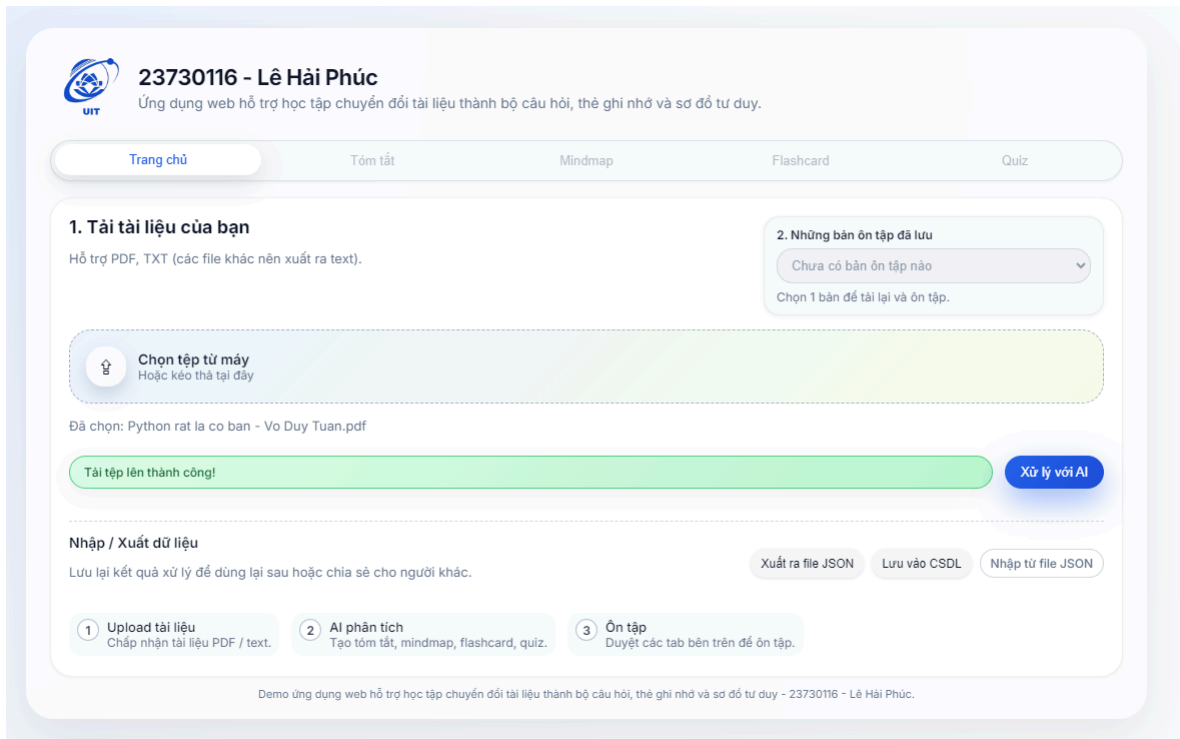
Dang chay app.py ...
(Neu muon dung lai, nhan Ctrl+C trong cua so nay)
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.6:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 887-894-109
192.168.1.6 - - [28/Nov/2025 10:26:20] "GET / HTTP/1.1" 200 -
192.168.1.6 - - [28/Nov/2025 10:26:20] "GET /styles.css HTTP/1.1" 200 -
192.168.1.6 - - [28/Nov/2025 10:26:20] "GET /script.js HTTP/1.1" 200 -
192.168.1.6 - - [28/Nov/2025 10:26:22] "GET /uit.png HTTP/1.1" 200 -
192.168.1.6 - - [28/Nov/2025 10:26:24] "GET /uit.png HTTP/1.1" 304 -
192.168.1.6 - - [28/Nov/2025 10:28:57] "POST /api/process_document HTTP/1.1" 200 -
```



Hình 4.4: Trang chủ sau khi khởi động và truy cập <http://localhost:5000>.

2. Tải tài liệu

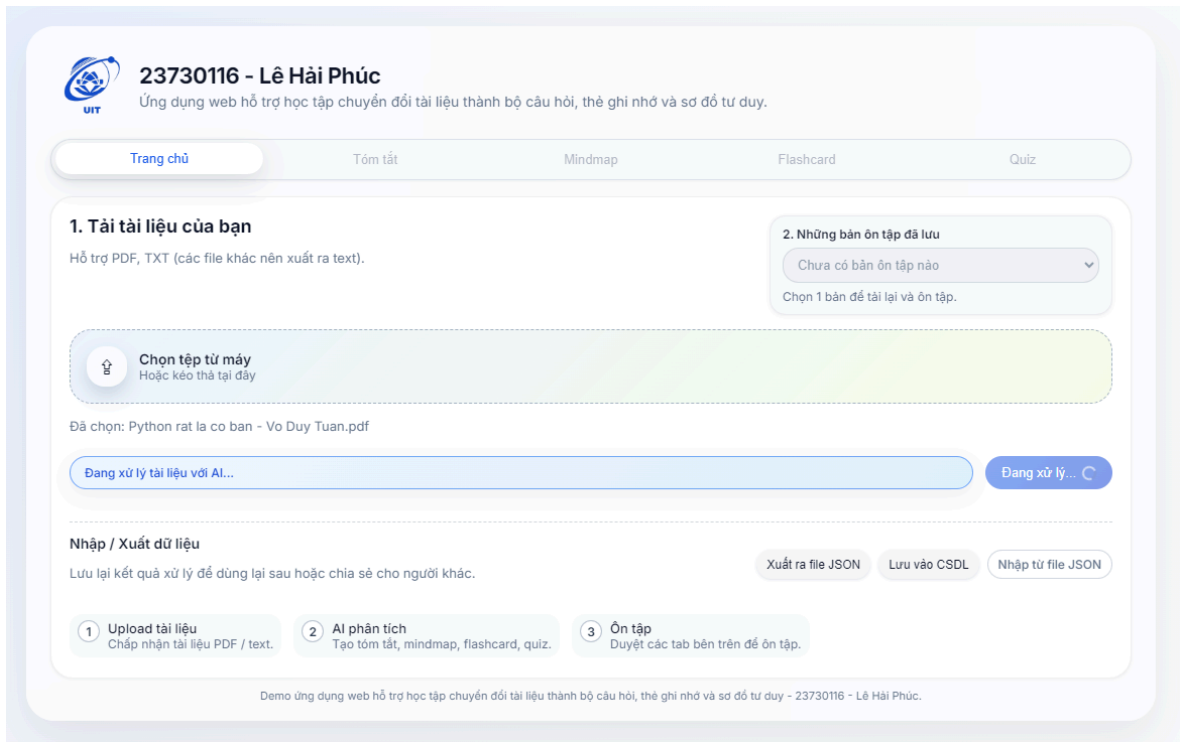
- Ở tab Trang chủ:
 - Bấm vào vùng chọn file hoặc kéo—thả tệp PDF/TXT vào.
 - Chờ trạng thái hiển thị “Tải tệp lên thành công!” (hoặc tương đương).



Hình 4.5: Trang chủ sau khi tải tệp lên thành công.

3. Xử lý với AI

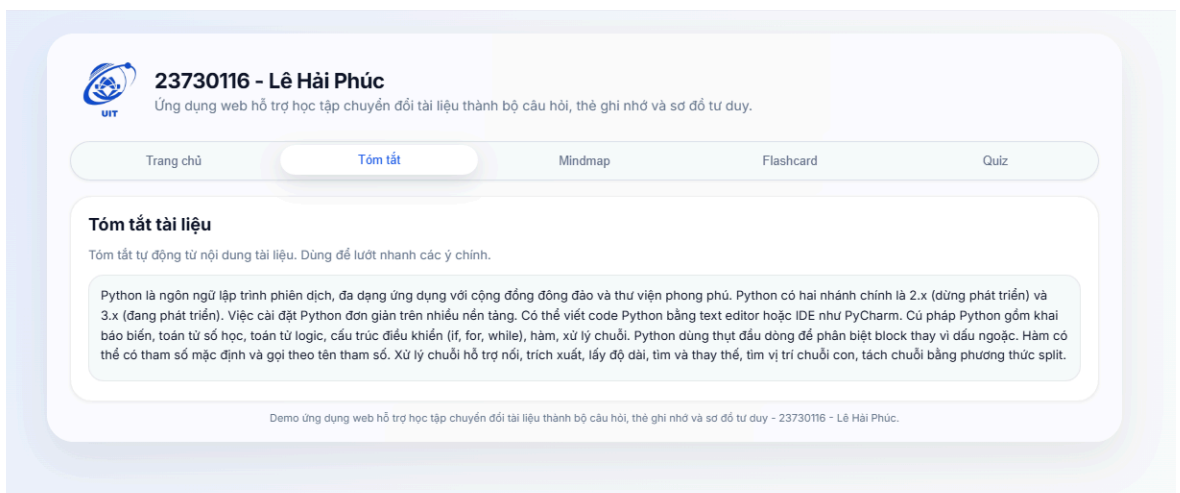
- Nhấn nút “Xử lý với AI”.
- Quan sát:
 - Nút đổi sang trạng thái loading, có spinner.
 - Dòng trạng thái hiển thị “Đang xử lý tài liệu với AI...”.
- Khi xong, trạng thái chuyển sang “Xử lý xong! Bạn có thể mở các tab để xem kết quả.” và các tab kết quả được mở khoá.



Hình 4.6: Trang chủ khi tải tệp lên thành công và nhấn nút “Xử lý với AI”.

4. Xem tóm tắt

- Vào tab Tóm tắt để xem bản rút gọn nội dung tài liệu.

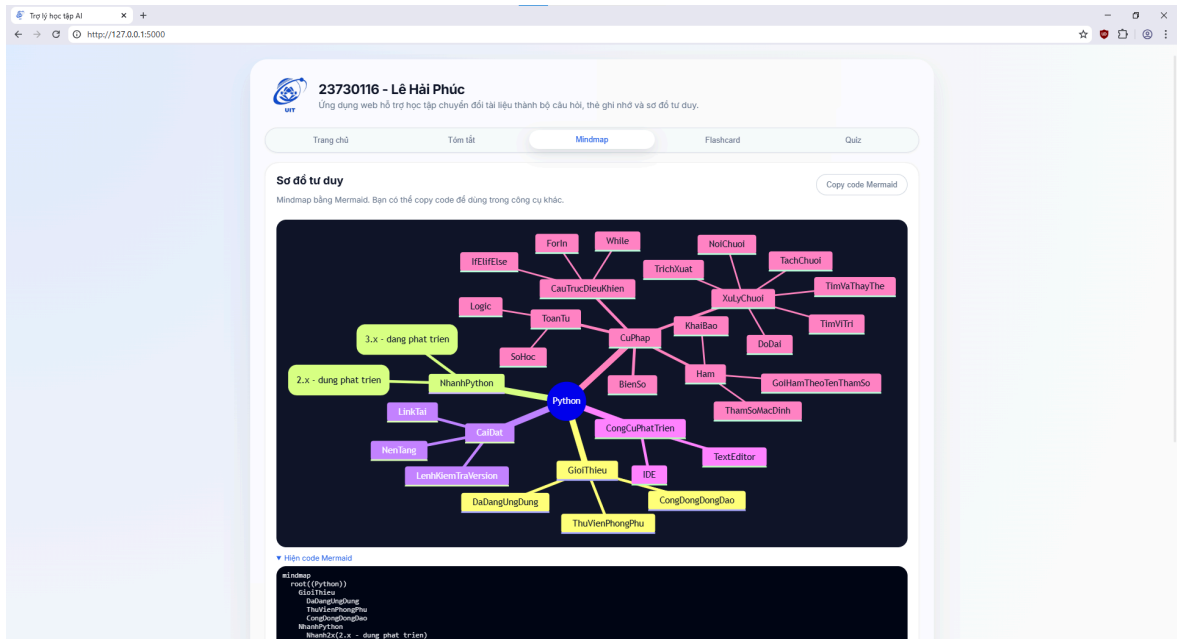


Hình 4.7: Tab Tóm tắt.

5. Xem sơ đồ tư duy

- Vào tab **Mindmap**:

- Xem sơ đồ đã render (nếu Mermaid tải thành công).
- Có thể bấm “Copy code Mermaid” để dùng code trong các công cụ khác.

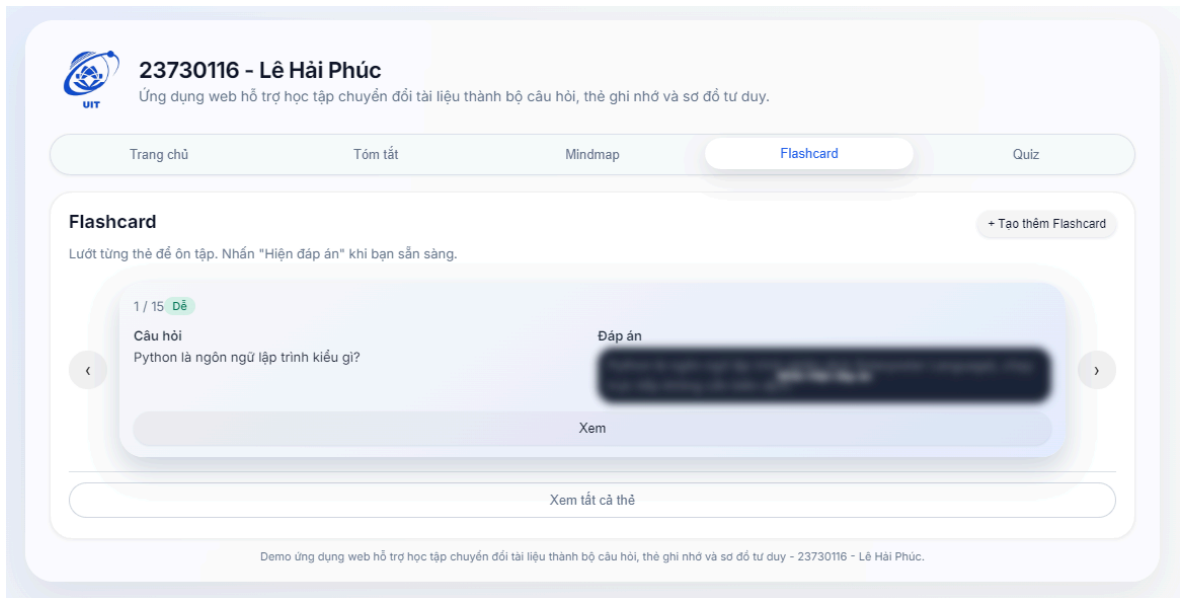


Hình 4.8: Tab Mindmap - sơ đồ tư duy.

6. Ôn tập bằng flashcard

- Vào tab Flashcard:

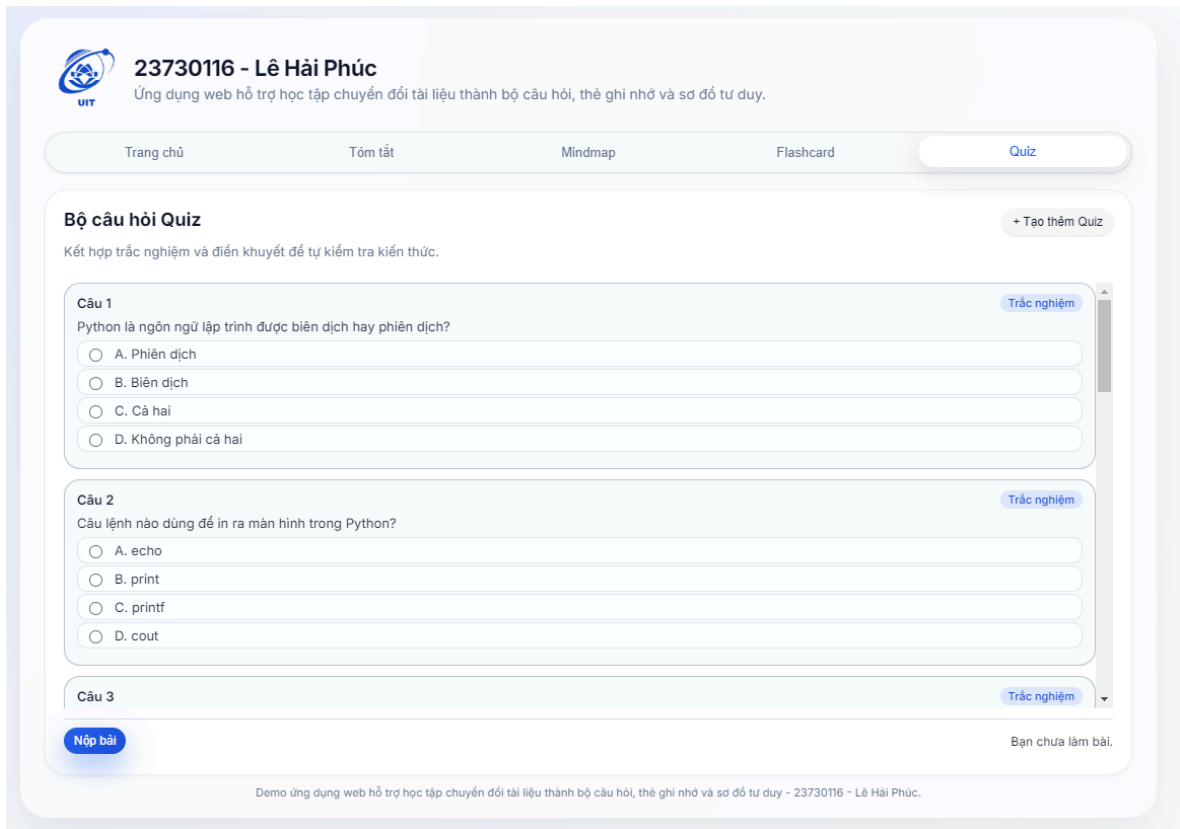
- Dùng < và > để chuyển giữa các thẻ.
- Bấm “Hiện/Ẩn đáp án” để tự kiểm tra.
- Quan sát badge độ khó trên mỗi thẻ.
- Nếu muốn xem toàn bộ, dùng chức năng “Xem tất cả thẻ”.
- Có thể tạo thêm Flashcard.



Hình 4.9: Tab Flashcard - thẻ ghi nhớ.

7. Làm bài quiz

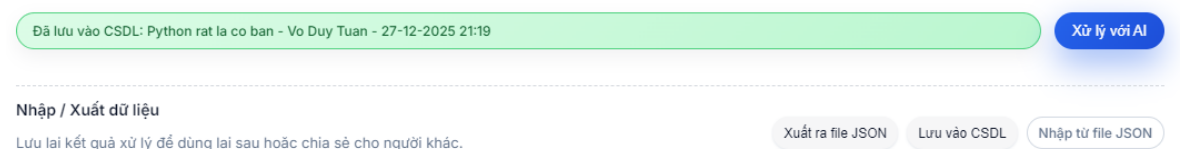
- Vào tab Quiz:
 - Trả lời từng câu (chọn radio hoặc điền đáp án).
 - Có thể tạo thêm câu hỏi.
 - Bấm “Nộp bài” để xem số câu đúng và đáp án chi tiết.



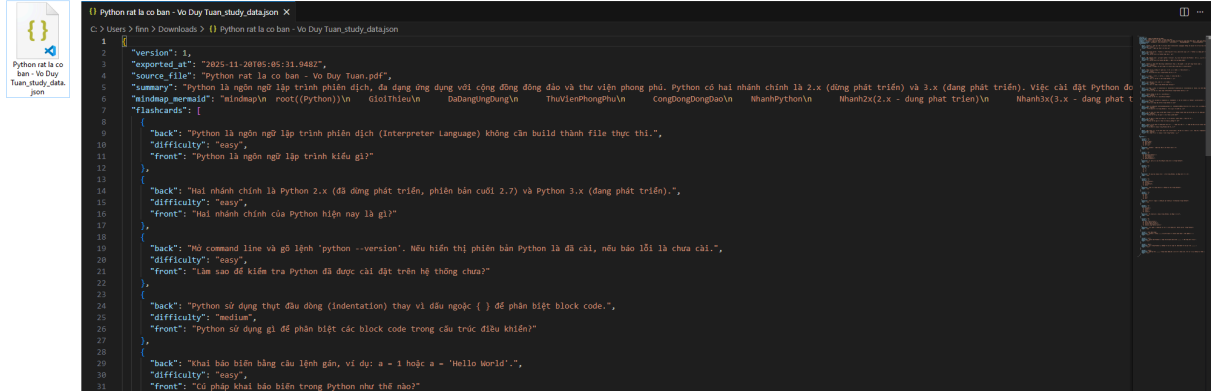
Hình 4.10: Tab Quiz - bộ câu hỏi.

8. Lưu / tải lại kết quả

- Sau khi hài lòng với bộ nội dung, có thể Lưu vào CSDL hoặc Xuất dữ liệu JSON để lưu lại.
- Khi cần, dùng chức năng Nhập dữ liệu từ JSON để tải lại mà không phải gọi lại AI.



Hình 4.11: Lưu vào CSDL



Hình 4.12: Tập dữ liệu JSON dùng để nhập/xuất dữ liệu.

Với Chương 4, báo cáo đã mô tả chi tiết cách cài đặt, triển khai, kiểm thử và sử dụng ứng dụng “Web hỗ trợ học tập: chuyển tài liệu thành tóm tắt, flashcard, quiz và sơ đồ tư duy”, bám sát chính xác cấu trúc mã nguồn, và hành vi runtime hiện tại của hệ thống.

CHƯƠNG 5. KẾT LUẬN

5.1. Tổng kết đóng góp và giá trị của đề tài

5.1.1. Giá trị học thuật và phương pháp

Từ góc độ học thuật, đồ án đã đi trọn vẹn một vòng đời phát triển ứng dụng web ở mức tương đối hoàn chỉnh: từ phân tích yêu cầu, đề xuất kiến trúc, thiết kế giao diện, lập trình backend/frontend đến tích hợp dịch vụ AI bên ngoài và xử lý lỗi.

Có thể tóm tắt các điểm nổi bật như sau:

(1) Vận dụng kiến thức lập trình web client–server

- **Backend (Flask)** xây dựng REST API `/api/process_document` với các chức năng:
 - Nhận file upload từ client.
 - Trích xuất văn bản từ PDF/TXT bằng PyPDF2 hoặc đọc text trực tiếp.
 - Gọi OpenAI Chat Completions API, nhận kết quả và trả JSON cho frontend.
- **Frontend (HTML/CSS/JavaScript thuần)**, thể hiện rõ tư duy:
 - Tách bạch giao diện: `index.html`, `styles.css`.
 - Tách bạch logic: `script.js`.
 - Quản lý trạng thái ứng dụng qua đối tượng `appState` (`summary`, `mindmap`, `flashcards`, `quizzes`, `source_file`).

(2) Ứng dụng mô hình ngôn ngữ lớn (LLM) một cách có kiểm soát

Đề án không chỉ “gọi API để sinh text” mà còn:

- Thiết kế một `SYSTEM_PROMPT` chi tiết, mô tả rõ vai trò “trợ lý học tập”, yêu cầu nội dung tiếng Việt và buộc mô hình trả về đúng một JSON hợp lệ.
- Định nghĩa schema JSON chuẩn gồm 4 trường:
 - `summary`
 - `mindmap_mermaid`
 - `flashcards`

- quizzes
với cấu trúc con rõ ràng, giúp việc parse và hiển thị phía client đơn giản, an toàn.
- Áp dụng xử lý bổ sung như `strip_json_code_fences()` để loại bỏ trường hợp mô hình bọc JSON trong json ... , một lỗi phổ biến khi làm việc với LLM.
- Giới hạn độ dài văn bản gửi lên (khoảng 8.000 ký tự) để phù hợp với giới hạn ngữ cảnh của mô hình và thời gian phản hồi.

(3) Thiết kế cấu trúc dữ liệu và luồng xử lý nhất quán

- Toàn bộ dữ liệu trao đổi giữa backend – frontend – mô hình AI đều thống nhất ở dạng **JSON**.
- Phía frontend:
 - `AppState` đóng vai trò Model trung tâm.
 - Các hàm `renderSummary()`, `renderMindmap()`, `renderFlashcard()`, `renderQuiz()` đóng vai trò View.
 - Các hàm xử lý upload, gọi API, import/export JSON đóng vai trò Controller.
→ Thể hiện tư duy kiến trúc tương tự MVC, dù không dùng framework MVC trực tiếp.

(4) Thực hành xử lý lỗi và bảo mật căn bản

- **Backend:**
 - Kiểm tra sự tồn tại và tính hợp lệ của file upload (field file, tên file).
 - Bắt lỗi trích xuất văn bản, lỗi parse JSON, lỗi khi gọi OpenAI; trả về thông báo rõ ràng cho client.

- Tách API key ra khỏi mã nguồn, nạp qua file .env bằng python-dotenv.

- **Frontend:**

- Kiểm tra trường hợp chưa chọn file thì không cho phép xử lý.
- Hiển thị trạng thái “Đang xử lý...”, “Xử lý xong” hoặc thông báo lỗi.

(5) Trải nghiệm đầy đủ quy trình làm đồ án phần mềm

- Phân tích được bài toán thực tế: “từ tài liệu → nội dung phục vụ học tập”.
- Đề xuất và hiện thực giải pháp tổng thể, không chỉ là một demo AI nhỏ lẻ.
- Tài liệu README, cấu trúc thư mục rõ ràng, kèm setup.bat và run_server.bat thể hiện năng lực đóng gói, triển khai và hướng dẫn sử dụng – những kỹ năng quan trọng trong học phần và công việc thực tế.

5.1.2. Giá trị thực tiễn của ứng dụng

Về mặt ứng dụng, hệ thống giải quyết một nhu cầu rất rõ ràng trong bối cảnh học tập hiện nay: học từ tài liệu số dày, nhiều chữ, ít cấu trúc hỗ trợ ôn tập.

(1) Hỗ trợ chuyển tài liệu thô thành “gói nội dung học tập”

Với vài thao tác đơn giản:

1. Upload một file PDF/TXT.
2. Nhấn “Xử lý với AI”.

Người học nhận được:

- Bản tóm tắt giúp nắm nhanh nội dung trọng tâm.

- Mindmap (code Mermaid) để hình dung cấu trúc kiến thức.
- Bộ flashcard để ôn tập dạng hỏi–đáp, hỗ trợ ghi nhớ.
- Bộ quiz trắc nghiệm, đúng/sai và điền khuyết để tự kiểm tra mức độ hiểu.

(2) Mọi thứ diễn ra trong trình duyệt, giao diện tiếng Việt, dễ thao tác

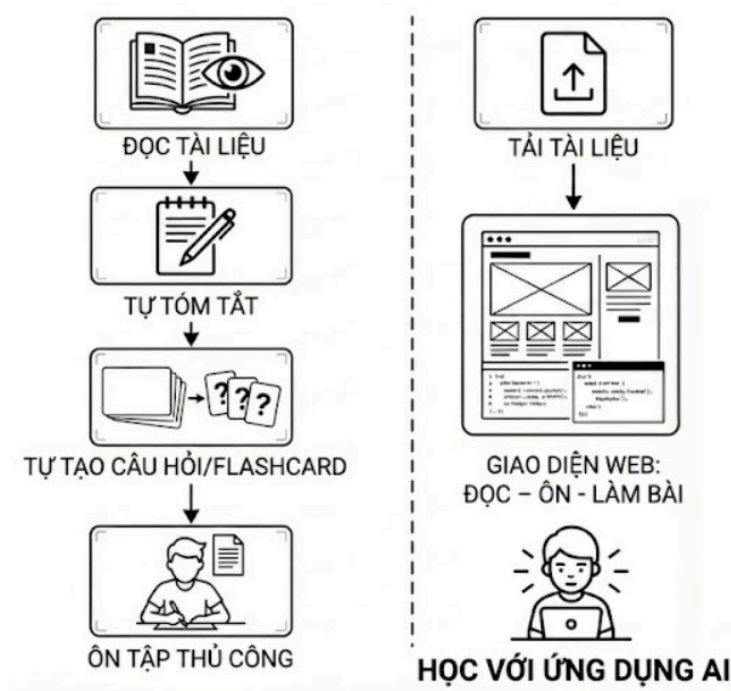
- Không yêu cầu cài đặt phần mềm nặng – chỉ cần trình duyệt và chạy server Flask local.
- Giao diện sáng, chia theo tab, sử dụng tiếng Việt, giúp sinh viên làm quen nhanh.

(3) Tạo nền tảng cho việc tái sử dụng kết quả học tập

- Chức năng Lưu vào CSDL hay Xuất / Nhập JSON cho phép:
 - Lưu trữ lại kết quả AI sinh ra để dùng nhiều lần.
 - Di chuyển bộ nội dung học tập giữa các máy khác nhau.
- Nhờ đó, ứng dụng không chỉ là công cụ “chạy một lần” mà là nền tảng xây dựng **kho nội dung học tập cá nhân**.

(4) Khả năng mở rộng sang nhiều tình huống sử dụng

- **Sinh viên:** ôn thi từng chương giáo trình, tạo flashcard/quiz cho mỗi chương.
- **Giảng viên / trợ giảng:** sinh bộ câu hỏi tham khảo từ tài liệu để dùng trong lớp.
- **Người tự học:** tổng hợp nội dung từ ebook, bài viết dài thành thẻ ghi nhớ và câu hỏi kiểm tra.



Hình 5.1: So sánh học tập thủ công và học cùng ứng dụng AI.

5.2. Hạn chế và hướng phát triển

5.2.1. Những hạn chế hiện tại

Trong phạm vi một đồ án môn học, việc giới hạn thời gian, tài nguyên và mục tiêu là điều tất yếu. Hệ thống hiện tại vẫn còn một số hạn chế, được nhìn nhận một cách trung thực như sau:

(1) Giới hạn độ dài ngữ cảnh

Hệ thống hiện giới hạn văn bản đầu vào theo số ký tự để đảm bảo ổn định và quy mô đồ án.

(2) Bảo mật và quản trị người dùng ở mức cơ bản

Hệ thống hiện tập trung demo chức năng:

- Chưa có đăng nhập/ phân quyền,
- Chưa có rate limit,

- Chưa mã hoá dữ liệu lưu trữ, nên nếu triển khai public sẽ cần nâng cấp.

(3) Chấm quiz còn đơn giản

- Fill-in hiện chấm theo so khớp chuỗi (dễ sai khi người dùng gõ khác cách viết).
- Chưa hỗ trợ đáp án đồng nghĩa, gần đúng, hoặc gợi ý theo mức.

5.2.2. Hướng phát triển trong tương lai

Phù hợp với định hướng “phát triển các ứng dụng web, tích hợp các kỹ thuật khai thác dữ liệu, gợi ý, Machine Learning”, hệ thống có thể được mở rộng theo các hướng sau:

(1) Xử lý tài liệu dài và nhiều tài liệu

- Áp dụng kỹ thuật chunking (chia nhỏ tài liệu), tóm tắt từng phần rồi tổng hợp lại (hierarchical summarization).
- Cho phép upload nhiều file cho cùng một chủ đề, sinh bộ tóm tắt/flashcard/quiz tổng hợp từ nhiều nguồn.

(2) Tích hợp OCR cho PDF scan

- Tự phát hiện PDF không có text layer,
- Tích hợp OCR (Tesseract hoặc dịch vụ OCR) để trích xuất chữ từ ảnh,
- Nâng độ phủ và tăng tính ứng dụng thực tế.

(3) Tích hợp các kỹ thuật khai thác dữ liệu và gợi ý

- Sử dụng dữ liệu về:
 - Câu hỏi nào người dùng thường trả lời sai.
 - Flashcard nào bị đánh dấu là “khó nhớ”.

- Từ đó:
 - Gợi ý thứ tự flashcard/quiz ưu tiên.
 - Đề xuất thêm câu hỏi hoặc nội dung cần ôn tập nhiều hơn.

(4) Cá nhân hoá học tập

- Chọn mức độ khó khi sinh quiz/flashcard,
- Theo dõi lịch sử làm quiz, thống kê lỗi sai,
- Gợi ý ôn tập theo điểm yếu (spaced repetition).

(5) Nâng cấp hệ thống dữ liệu

- Bổ sung bảng người dùng, phân quyền, đồng bộ.
- Cho phép tag/nhãn (môn học, chương, ngày thi),
- Tìm kiếm trong “bản ôn tập đã lưu”.

(6) Cải tiến chấm điểm quiz

- Fill-in: so khớp linh hoạt (normalize, bỏ dấu, chấp nhận từ đồng nghĩa),
- MCQ: phân tích câu nhiễu, phát hiện câu quá dễ,
- TF: tạo mệnh đề chặt, tránh mơ hồ.

(7) Đa dạng hoá mô hình và tùy chọn cấu hình

- Cho phép chọn mô hình AI (ví dụ: mô hình rẻ hơn để sinh nhanh, mô hình mạnh hơn để sinh nội dung chất lượng hơn).
- Tách phần cấu hình số lượng flashcard/quiz, kiểu quiz (trắc nghiệm, điền khuyết, kết hợp) ra thành **form** trên giao diện, thay vì “cố định trong prompt”.

(8) Nâng cấp giao diện và trải nghiệm người dùng

- Tối ưu UX trên mobile, hỗ trợ thao tác chạm với flashcard (vuốt trái/phải để chuyển thẻ).
- Thêm chế độ “học xen kẽ”: hiển thị xen kẽ tóm tắt – flashcard – quiz trong một flow học liền mạch thay vì tách tab.
- Hỗ trợ dark mode, điều chỉnh cỡ chữ để thân thiện với người đọc lâu.

(6) Mở rộng ngôn ngữ và lĩnh vực áp dụng

- Hiện tại, prompt ưu tiên tiếng Việt nếu tài liệu là tiếng Việt. Trong tương lai có thể:
 - Bổ sung lựa chọn ngôn ngữ output (Việt–Anh).
 - Cho phép tùy chọn “giải thích chi tiết hơn” cho các môn kỹ thuật (Toán, Lập trình) hoặc “tập trung vào ví dụ” cho các môn xã hội.

5.3. Kết luận chung

Đồ án đã hoàn thành mục tiêu xây dựng một ứng dụng web hỗ trợ học tập có khả năng chuyển đổi tài liệu PDF/TXT thành bộ học liệu đa dạng, hỗ trợ ôn tập tương tác, và đặc biệt có khả năng lưu–tải để tái sử dụng thông qua JSON và CSDL. Hệ thống thể hiện rõ tư duy thiết kế theo hướng kỹ thuật phần mềm: có phân tích yêu cầu, thiết kế kiến trúc, thiết kế dữ liệu, thiết kế API, kịch bản kiểm thử và đánh giá theo tiêu chí định lượng.

Dù vẫn còn các hạn chế về xử lý tài liệu dài, PDF scan, và kiểm soát chất lượng đầu ra, đề tài đã tạo ra một nền tảng vững để phát triển thành sản phẩm hoàn thiện hơn trong tương lai, đặc biệt khi bổ sung các cơ chế nâng cao như chunking, OCR, ràng buộc schema mạnh, trích dẫn nguồn và cá nhân hoá quá trình ôn tập.

Với những gì đã được thiết kế và hiện thực, đồ án không chỉ dừng lại ở việc hoàn thành một bài tập lớn, mà còn đặt nền móng cho một trợ lý học tập AI có tiềm

năng phát triển thành sản phẩm hữu ích trong thực tế. Kinh nghiệm thu được từ việc phân tích bài toán, tích hợp LLM, thiết kế kiến trúc và giao diện sẽ là hành trang quan trọng để tiếp tục nghiên cứu, mở rộng và xây dựng các hệ thống web thông minh hơn trong tương lai.

TÀI LIỆU THAM KHẢO

[1] Thạc Bình Cường, *Phân tích và thiết kế hệ thống thông tin*, Nhà xuất bản Bách Khoa Hà Nội, Hà Nội, 2023. Truy cập ngày 10/11/2025, tại:

<https://nxbbachkhoa.vn/phan-tich-thiet-ke-he-thong-thong-tin-b10784.html>

[2] Nguyễn Đăng Khoa; Nguyễn Quốc Hưng, *Phát triển ứng dụng trí tuệ nhân tạo (AI) trong giáo dục đại học tại Việt Nam: cơ hội, thách thức và giải pháp*, Khoa Thống kê – Trường Công nghệ – Trường Đại học Kinh tế Quốc dân, 2025. Truy cập ngày 17/11/2025, tại:

<https://daotao.neu.edu.vn/Resources/Docs/SubDomain/daotao/CDR/10-TS-NguyenDangKhoa-NguyenQuocHung.pdf> Daotao

[3] Pallets Projects, *Flask Documentation (Version 3.1.x)*, Flask Official Documentation, 2025. Truy cập ngày 25/11/2025, tại:

<https://flask.palletsprojects.com>

[4] OpenAI, *OpenAI API Reference – Chat Completions*, OpenAI Platform, 2025. Truy cập ngày 28/11/2025, tại: <https://platform.openai.com/docs/api-reference/chat>

[5] PyPDF2 Developers, *PyPDF2 Documentation*, 2023. Truy cập ngày 02/12/2025, tại: <https://pypdf2.readthedocs.io/> PyPDF2

[6] MermaidJS, *Mermaid: Diagramming and Charting Tool – Official Documentation*, 2025. Truy cập ngày 05/12/2025, tại: <https://mermaid.js.org/Mermaid>